

Contextual Word Representations, Transformers, and BERT

Text and Web Mining (IS6751)

School of Communication and Information

Representations for a word

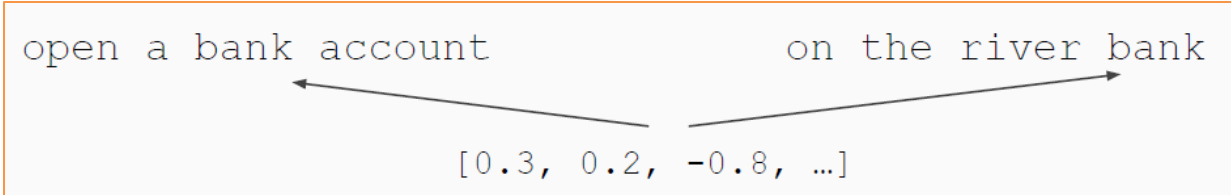
- Up until now, we've basically had **one representation** of words:
 - The word vectors that we learned about
 - Word2vec and GloVe
- This causes problems:
 - Always the same representation for a **word type** regardless of the context in which a **word token** occurs.
 - But words have different **aspects**, including semantics and syntactic behavior.
 - We might want very fine-grained word sense disambiguation: **Contextual word representations**

Bank (1): an organization that provides various financial services.
Bank (2): the side of a river, canal, etc. and the land near it.

open a bank account

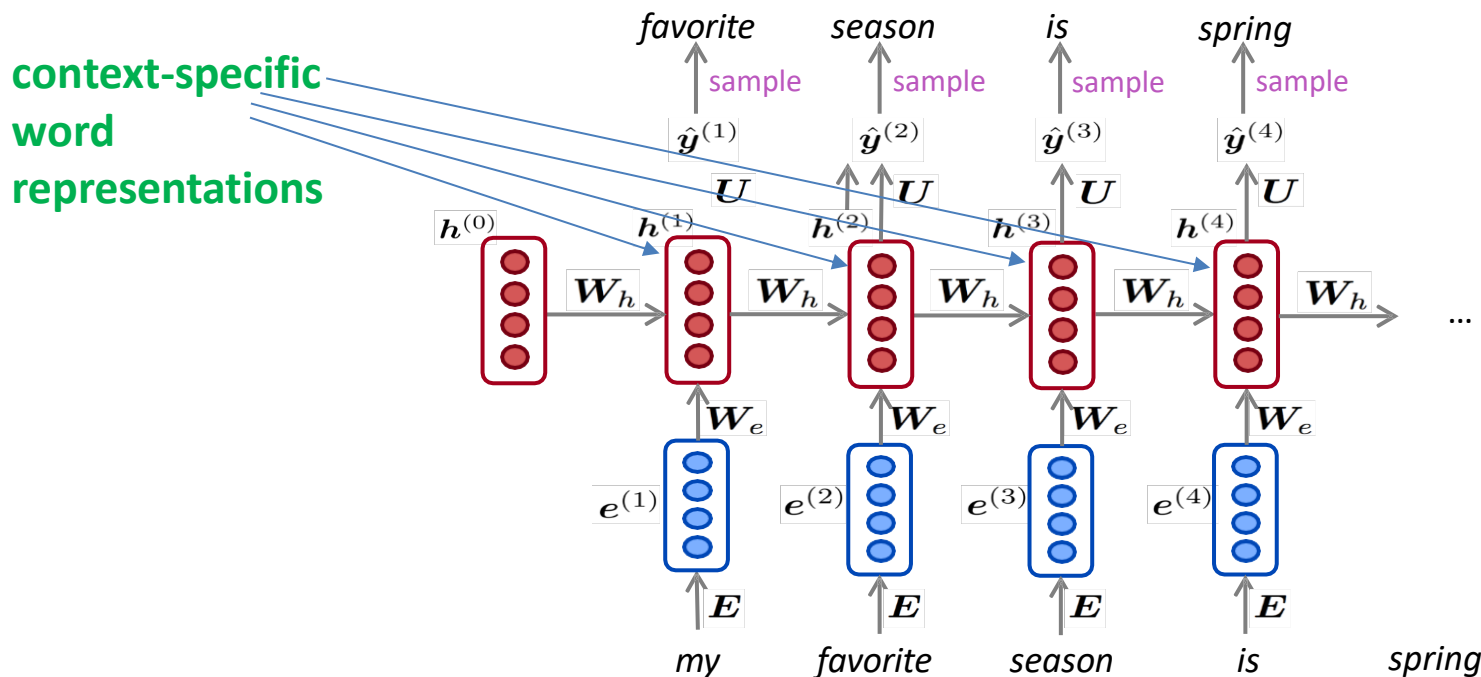
on the river bank

[0.3, 0.2, -0.8, ...]



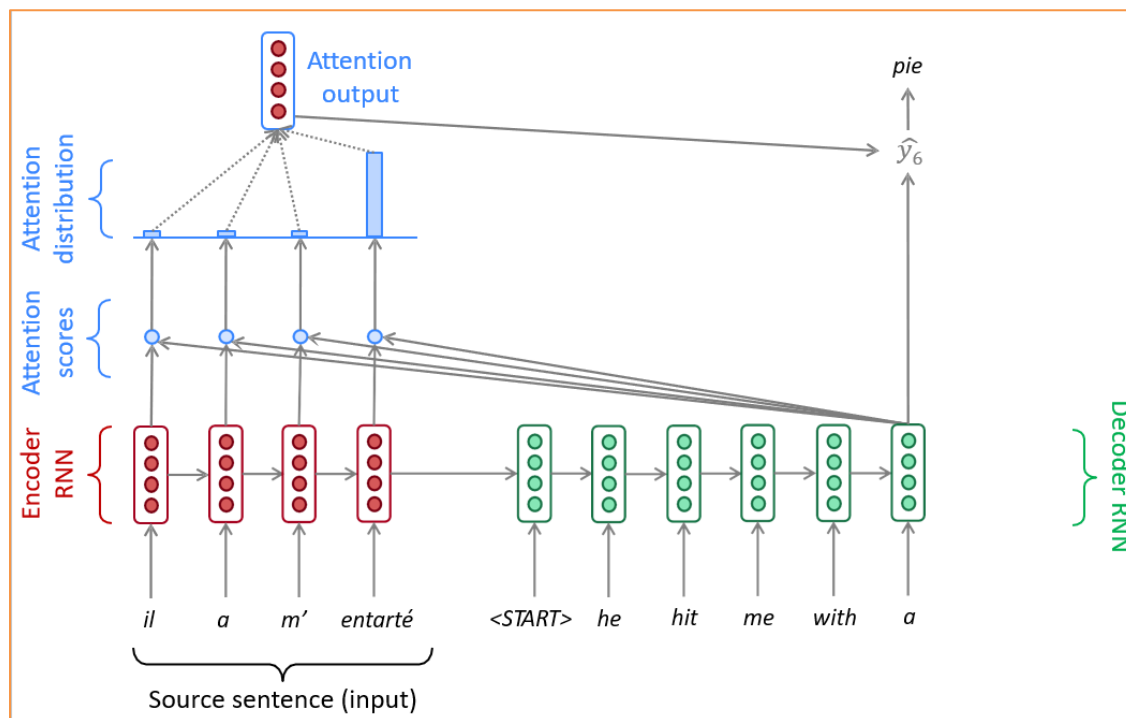
Did we all along have a solution to this problem?

- In a Neural LM (Language Model), we use word vectors through LSTM (or RNN) layers.
- Those LSTM layers are trained to predict the next word.
- But those language models are producing **context-specific word representations** at each position!

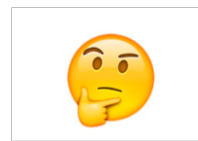


The Motivation for Transformers

- We want **parallelization** but RNNs are inherently sequential.



- Despite GRUs and LSTMs, RNNs still need **attention mechanism** to deal with long range dependencies.
- But if **attention** gives us access to any state... maybe we can just use attention and don't need the RNN?



The Transformer

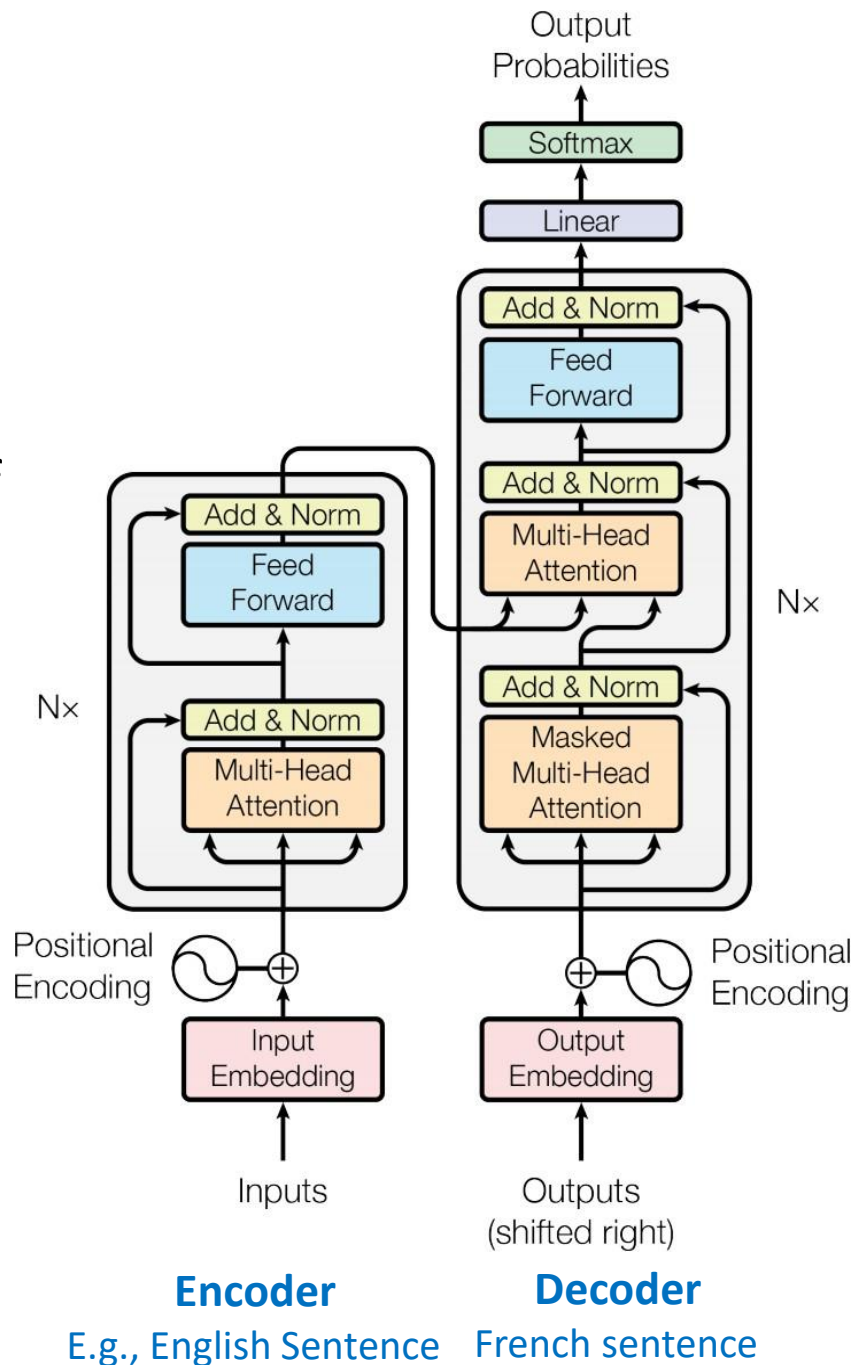
Attention is all you need. 2017.

Vaswani, Shazeer, Parmar, Uszkoreit,

Jones, Gomez, Kaiser, Polosukhin

<https://arxiv.org/pdf/1706.03762.pdf>

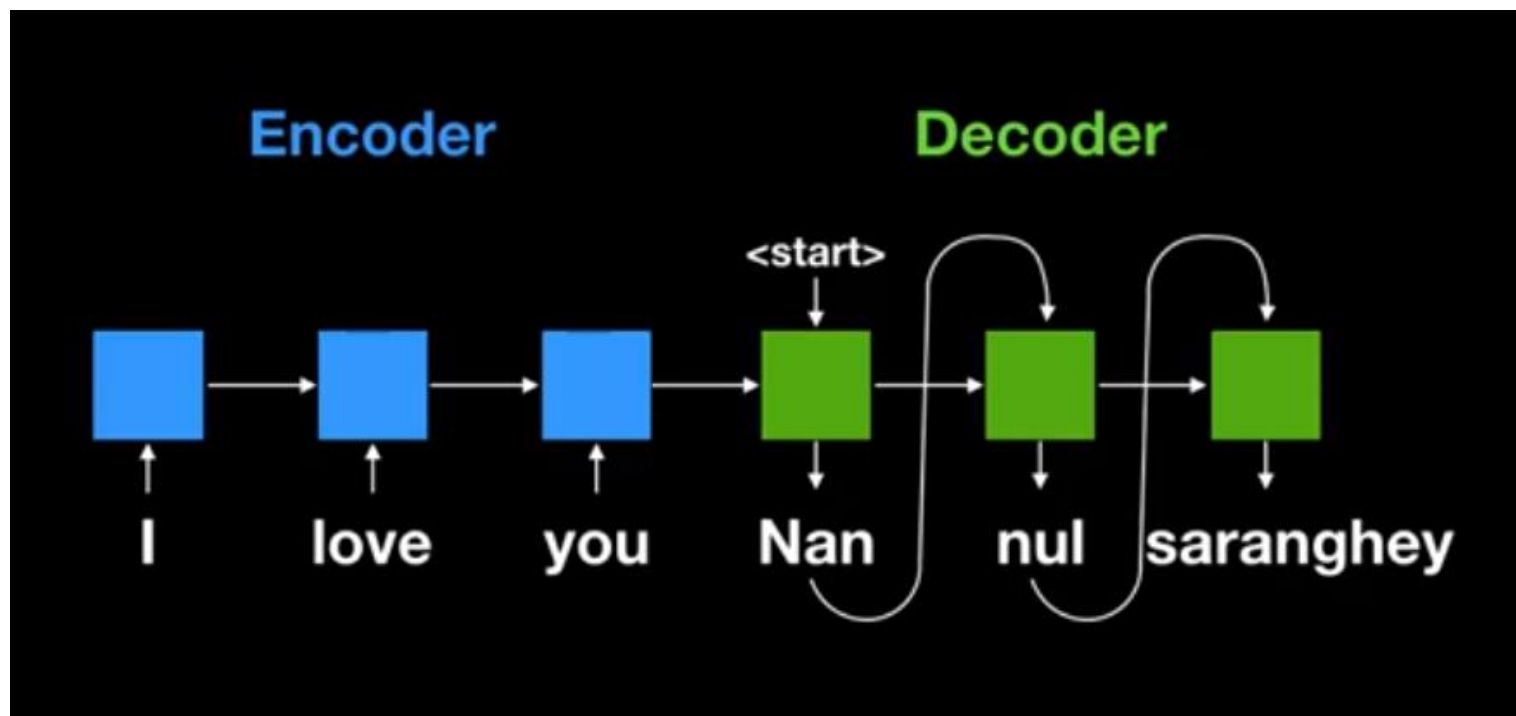
- **Non-recurrent** sequence-to-sequence encoder-decoder model
- Task: machine translation with parallel corpus
- Predict each translated word
- Final cost/error function is standard cross-entropy error on top of a softmax classifier



The Transformer

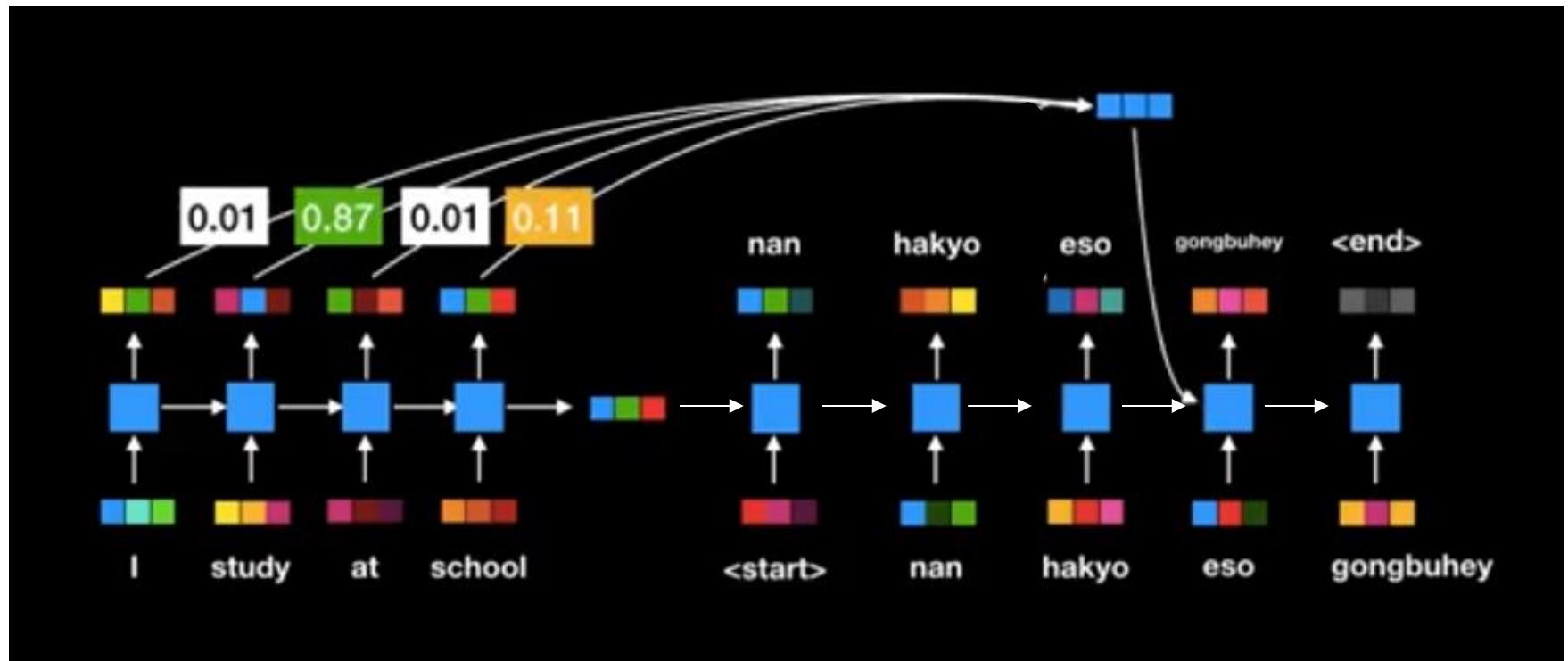
RNN-based encoder decoder

- Translate a sentence in **English** to a sentence in **Korean**.

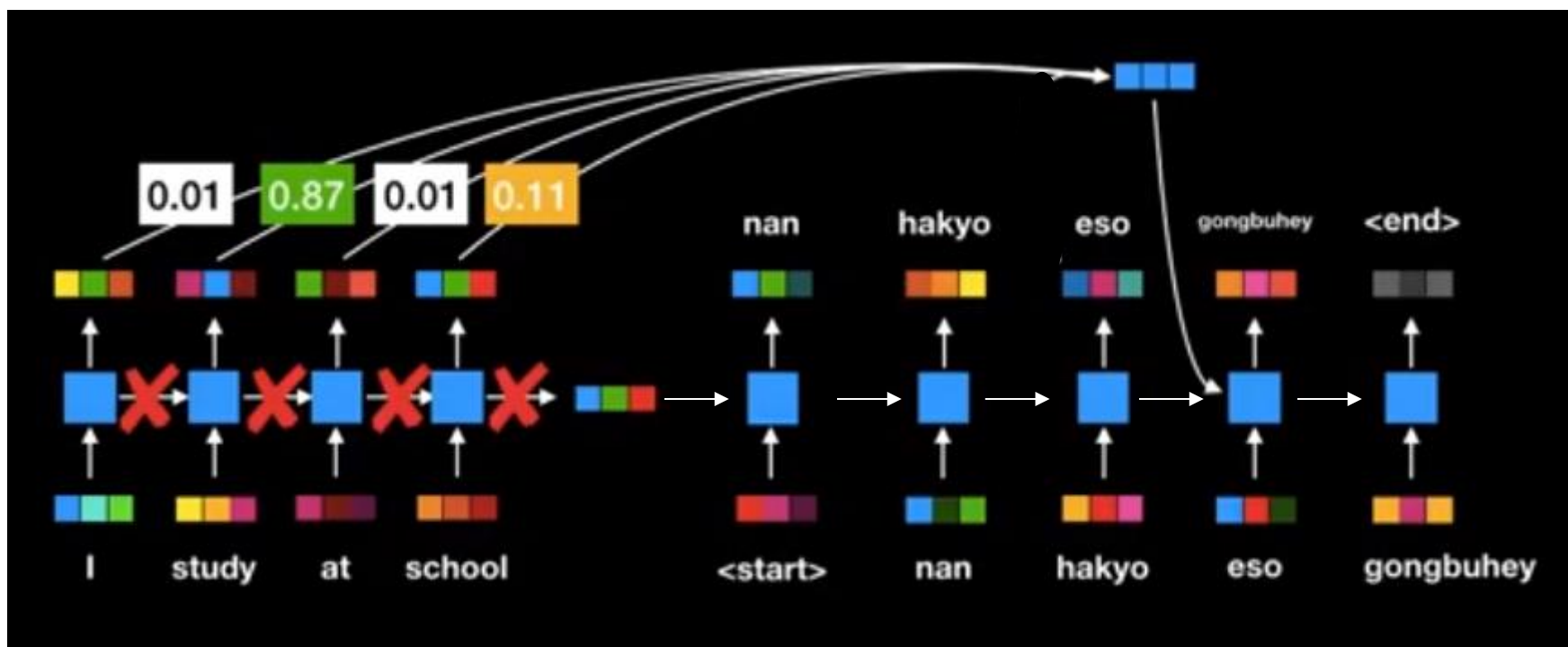


Source: Transformer, Minsuk Heo,
<https://www.youtube.com/watch?v=z1xs9jdZnuY>

RNN-based encoder decoder with attention

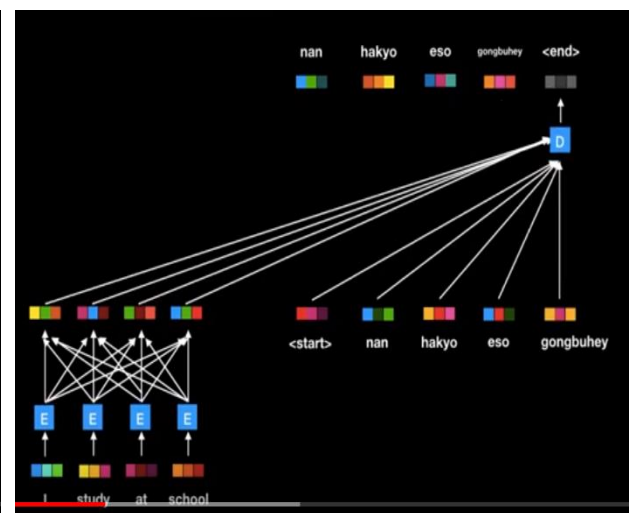
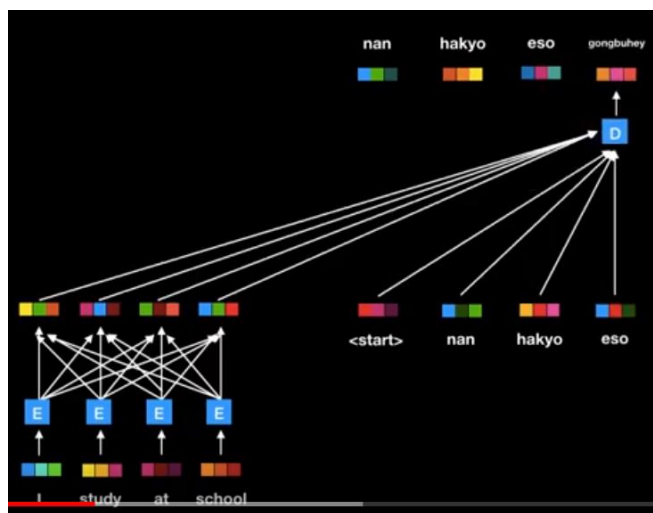
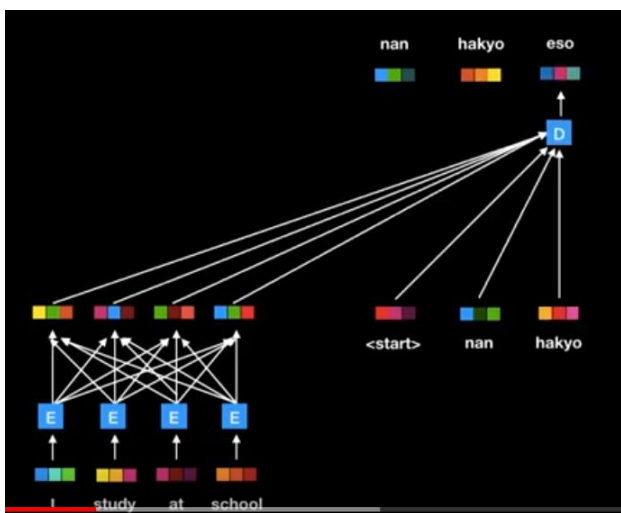
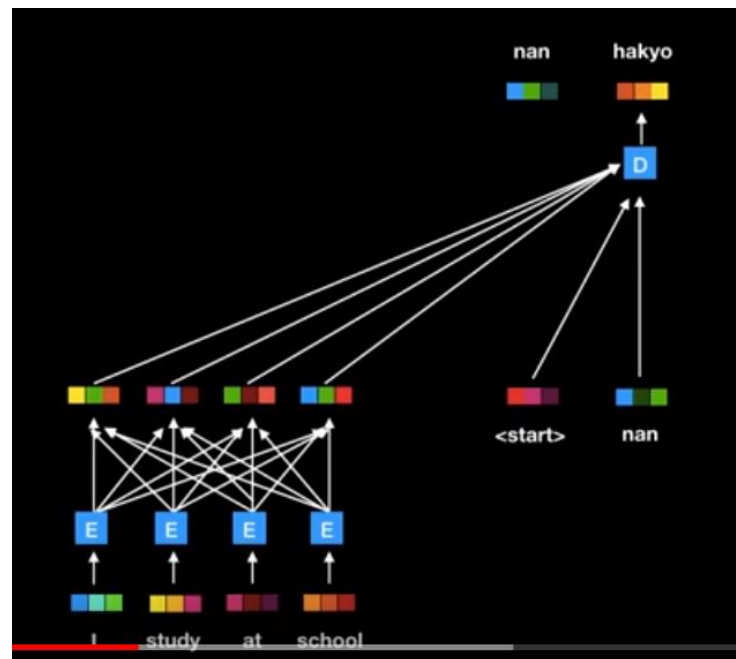
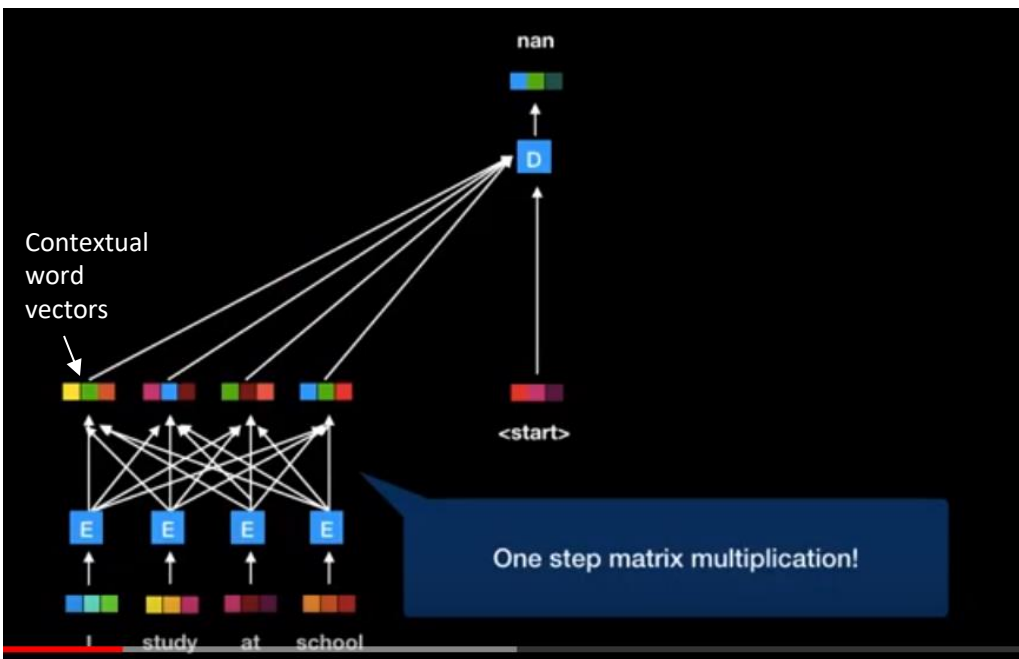


Can we remove RNN, and just use attention?

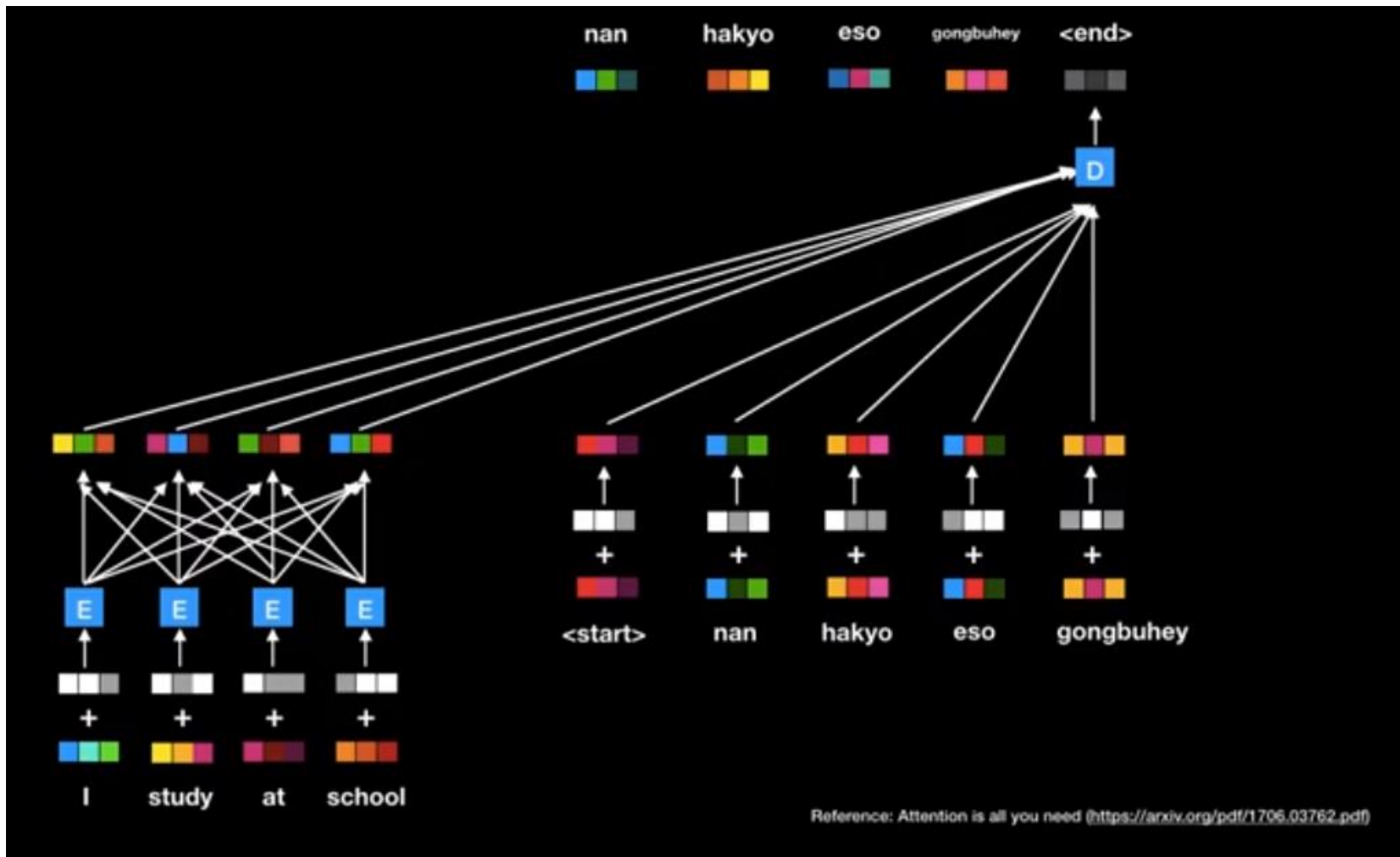


Yes, attention is all we need!

E: Encoder; D: Decoder



Positional encoding: word order is important



How the attention works:

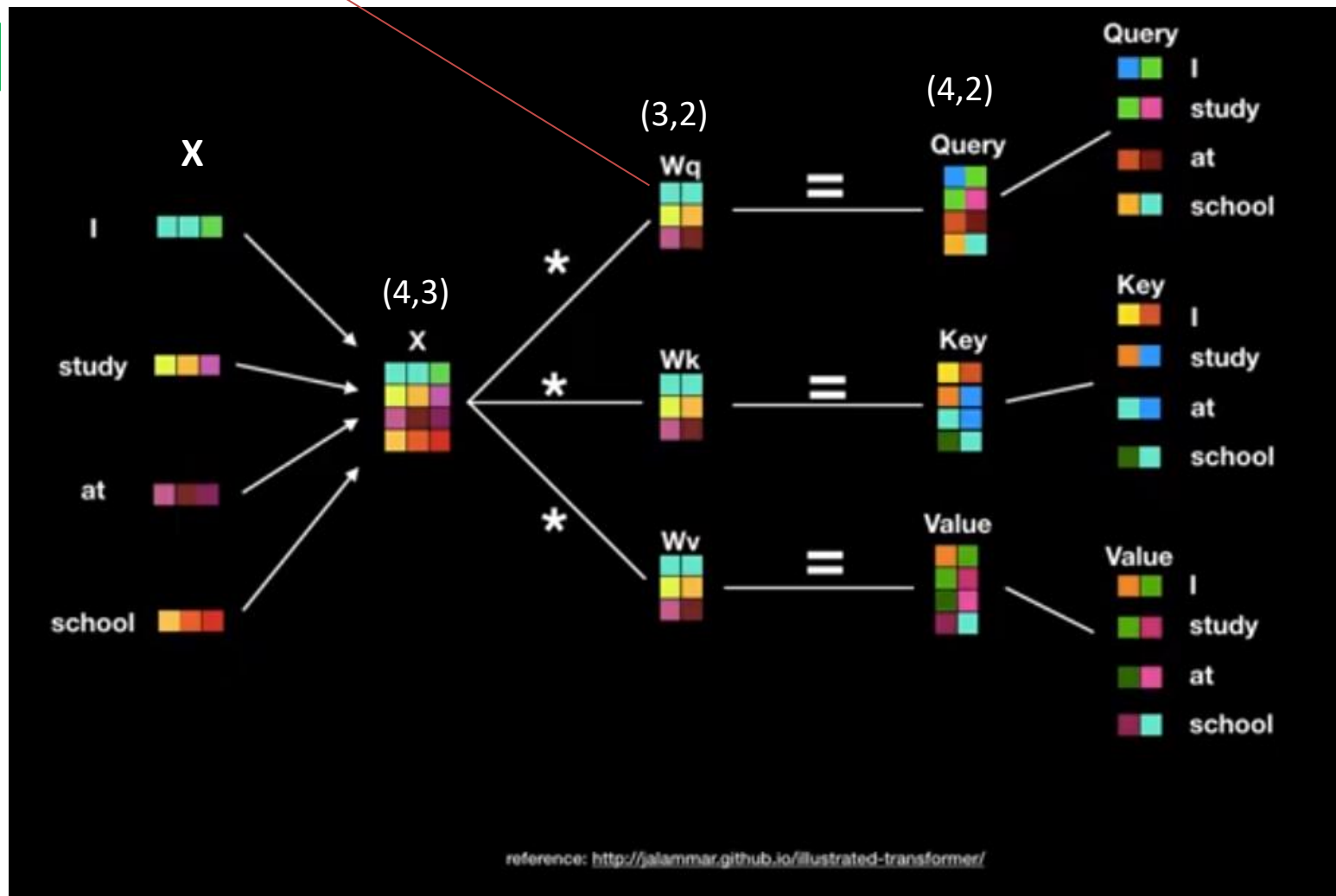
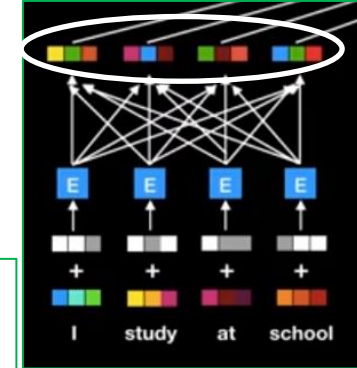
Self Attention

```
self.query = nn.Linear (3, 2, bias=False)  
query = self.query (x)  # query  $\in \mathbb{R}^2$ 
```

$x \in \mathbb{R}^3$

```
self.key = nn.Linear (3, 2, bias=False)  
key = self.key (x)  # key  $\in \mathbb{R}^2$ 
```

```
self.value = nn.Linear (3, 2, bias=False)  
value = self.value (x)  # value  $\in \mathbb{R}^2$ 
```



Self Attention

	X	Query	Key	Value
I				
study				
at				
school				

Transformer (Attention is all you need)

- When **Query term** is "I":
 - Initially, I*I score is the highest.

Attention scores

	X	Query	Key	Value
I				
study				
at				
school				

	Query * Key ^T	Score	Softmax
I	I * I * = 130		0.92
	I * study * = 50		0.05
	I * at * = 20		0.02
	I * school * = 10		0.01

Transformer (Attention is all you need)

- When Query term is “I”:
 - Eventually, “I” will focus on more meaningful words, like **study** and **school**.

	X	Query	Key	Value
I				
study				
at				
school				

Attention scores

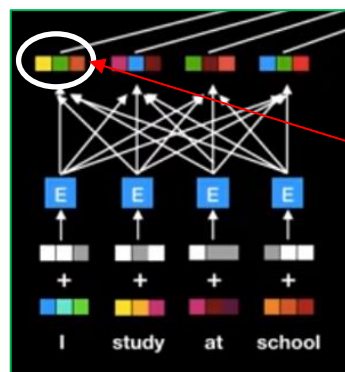
Transformer (Attention is all you need)

	Score
I * I	130
I * study	50
I * at	20
I * school	10

Softmax
0.92
0.05
0.02
0.01

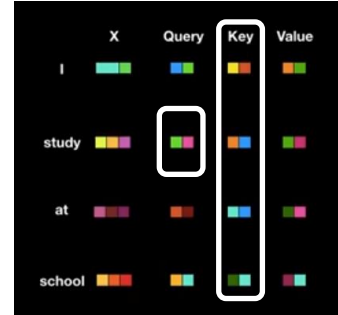
Value
I
study
at
school

Softmax * Value	$\sum$ Softmax * Value (Attention layer output)



Contextual word vector of “I”.
(actually, needs more steps to get this contextual vector.)

When Query term is
"study":

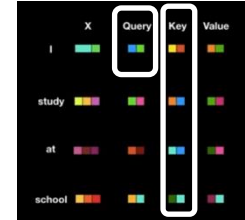


Transformer (Attention is all you need)

		Query * Key ^T	Score	Softmax	Value	Softmax * Value	Σ Softmax * Value (Attention layer output)
I	I * I	 *  = 130	0.92		I		
	I * study	 *  = 50	0.05		study		
	I * at	 *  = 20	0.02		at		
	I * school	 *  = 10	0.01		school		
study	study * I	 *  = 30	0.02		I		
	study * study	 *  = 110	0.70		study		
	study * at	 *  = 20	0.03		at		
	study * school	 *  = 70	0.25		school		
at	at * I	 *  = 30	0.03		I		
	at * study	 *  = 50	0.10		study		
	at * at	 *  = 90	0.80		at		
	at * school	 *  = 40	0.07		school		
school	school * I	 *  = 30	0.01		I		
	school * study	 *  = 80	0.27		study		
	school * at	 *  = 23	0.02		at		
	school * school	 *  = 160	0.70		school		

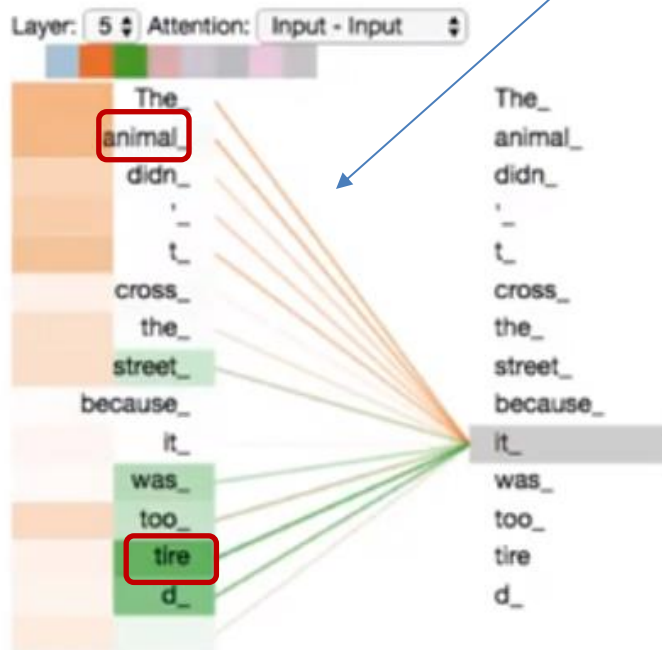
Contextual
word
vectors of
"I study at
school"

Transformer (Attention is all you need)



When Query term is “I”
in “I study at school”:
- focuses on “I”.

Transformer (Attention is all you need) ^T				Score	Softmax	Value	Softmax * Value	Σ Softmax * Value (Attention layer output)
I	I * I		*		= 130	0.92	I	
	I * study		*		= 50	0.05	study	
	I * at		*		= 20	0.02	at	
	I * school		*		= 10	0.01	school	



When query term is “it”

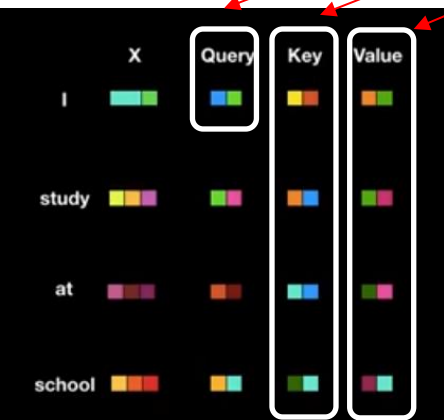
- “it (query vector) * animal (key vector)” and “it * tire” get high attention scores.
- So, focuses on both animal and tire.

As we encode the word “it”, one attention head is focusing most on “the animal”, while another is focusing on “tired” – in a sense, the model’s representation of the word “it” bakes in some of the representation of both “animal” and “tired”.

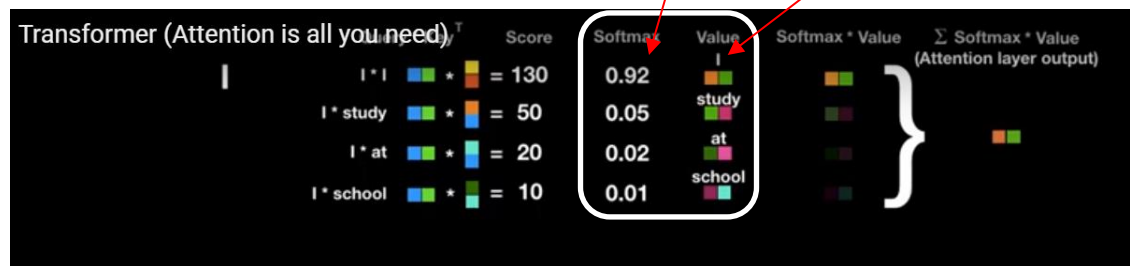
Dot-Product Attention (Equation): Contextual Vector

- Inputs: a query q and a set of key-value ($k-v$) pairs.
- Output is **weighted sum of values**, where
- Weight of each value is computed by an inner product of query and corresponding key.
- Queries and keys have same dimensionality, d_k value.

$$A(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$



When Query term is "I":



Dot-Product Attention – Matrix notation

- When we have multiple queries $\mathbf{q}$, we stack them in a matrix $\mathbf{Q}$:

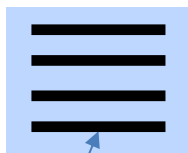
	X	Query	Key	Value
I				
study				
at				
school				

$$A(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

- Becomes: $A(Q, K, V) = \text{softmax}(QK^T)V$

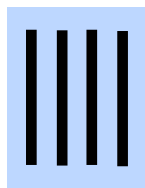
$$[|Q| \times d_k] \times [d_k \times |K|] \times [|K| \times d_v]$$

softmax
row-wise



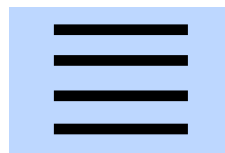
E.g., 4×2

x



2×4

x



4×2

$$= [|Q| \times d_v]$$

	Query * Key ^T	Score	Softmax	Value	Softmax * Value	Σ Softmax * Value (Attention layer output)
I	I * I = 130	0.92		I		
I * study	= 50	0.05		study		
I * at	= 20	0.02		at		
I * school	= 10	0.01		school		
study	study * I = 30	0.02		I		
study * study	= 110	0.70		study		
study * at	= 20	0.03		at		
at	at * I = 30	0.03		I		
at * study	= 50	0.10		study		
at * at	= 90	0.80		at		
school	school * I = 30	0.01		I		
school * study	= 80	0.27		study		
school * at	= 23	0.02		at		
school * school	= 160	0.70		school		

Scaled Dot-Product Attention

- Problem: As d_k (dimension of key) gets large, the value of $\text{Query} * \text{Key}^T$ increases -> some values inside the softmax get large -> so the Transformer rescales the dot product to prevent it from exploding into huge values.
- Solution: Scale by length of query/key vectors:

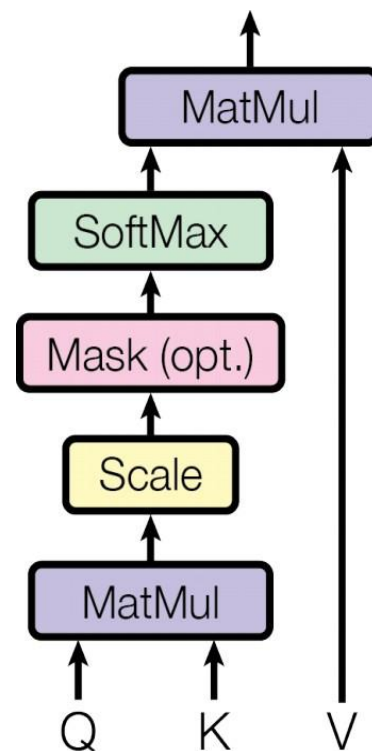
$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Input: "I study at school <padding> <padding> <padding>."

When Query term is "I":

	Query * Key ^T	Score	Softmax
I	I * I = 130	0.92	
I * study	I * study = 50	0.05	
I * at	I * at = 20	0.02	
I * school	I * school = 10	0.01	
I * padding	= 0	0.00	
I * padding	= 0	0.00	
I * padding	= 0	0.00	

$$\text{softmax}\left(\frac{\text{Score}}{\text{sqrt}(\text{dimension of key})}\right)$$

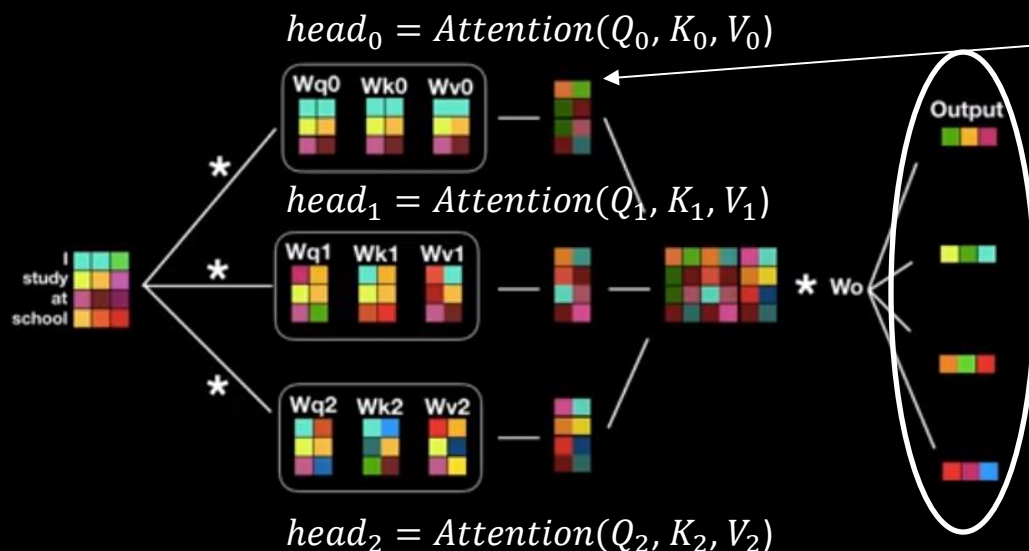


Attention Mask, [1, 1, 1, 1, 0, 0, 0], used to set scores with padding words to zero: E.g., set "I * padding" to 0.

Transformer (Attention is all you need)

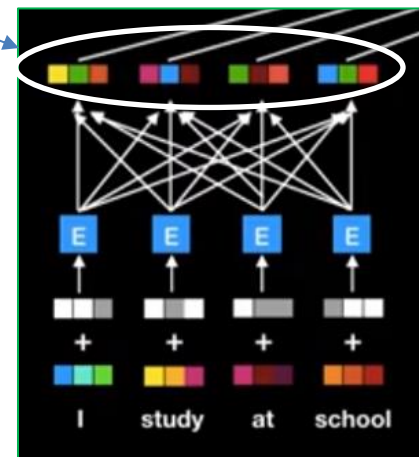
- Problem with simple (i.e., one head) self-attention:
- Only one way for words to interact with one-another.
- Solution: **Multi-head attention**

Multi Head Attention

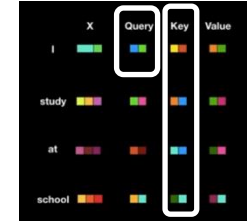


reference: <http://jalammar.github.io/illustrated-transformer/>

	Query * Key ^T	Score	Softmax	Value	Softmax * Value	Σ Softmax * Value (Attention layer output)
I	I * I = 130	0.92				
I	I * study = 50	0.05				
I	I * at = 20	0.02				
I	I * school = 10	0.01				
study	study * I = 30	0.02				
study	study * study = 110	0.70				
study	study * at = 20	0.03				
study	study * school = 70	0.25				
at	at * I = 30	0.03				
at	at * study = 50	0.10				
at	at * at = 90	0.80				
at	at * school = 40	0.07				
school	school * I = 30	0.04				
school	school * study = 80	0.27				
school	school * at = 23	0.02				
school	school * school = 160	0.70				

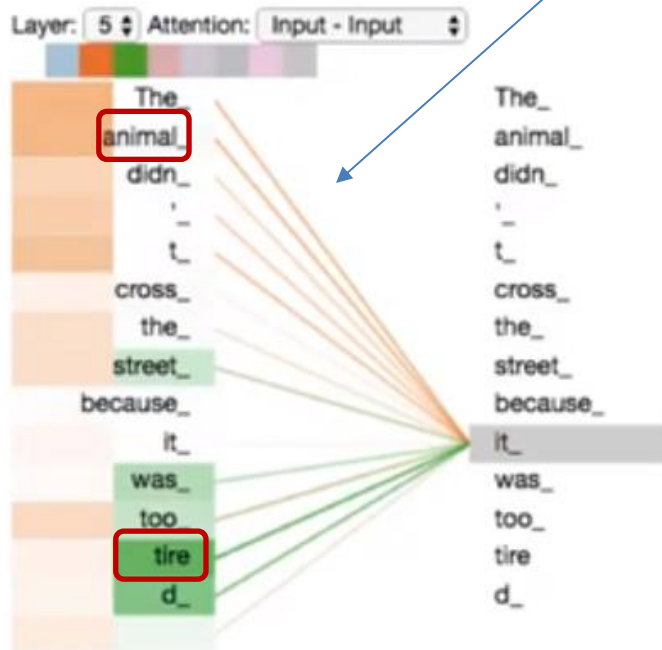


Transformer (Attention is all you need)



When Query term is “I”
in “I study at school”:
- focuses on “I”.

Transformer (Attention is all you need) ^T				Score	Softmax	Value	Softmax * Value	Σ Softmax * Value (Attention layer output)
I	I * I		=	130	0.92	I		}
	I * study		=	50	0.05	study		
	I * at		=	20	0.02	at		
	I * school		=	10	0.01	school		



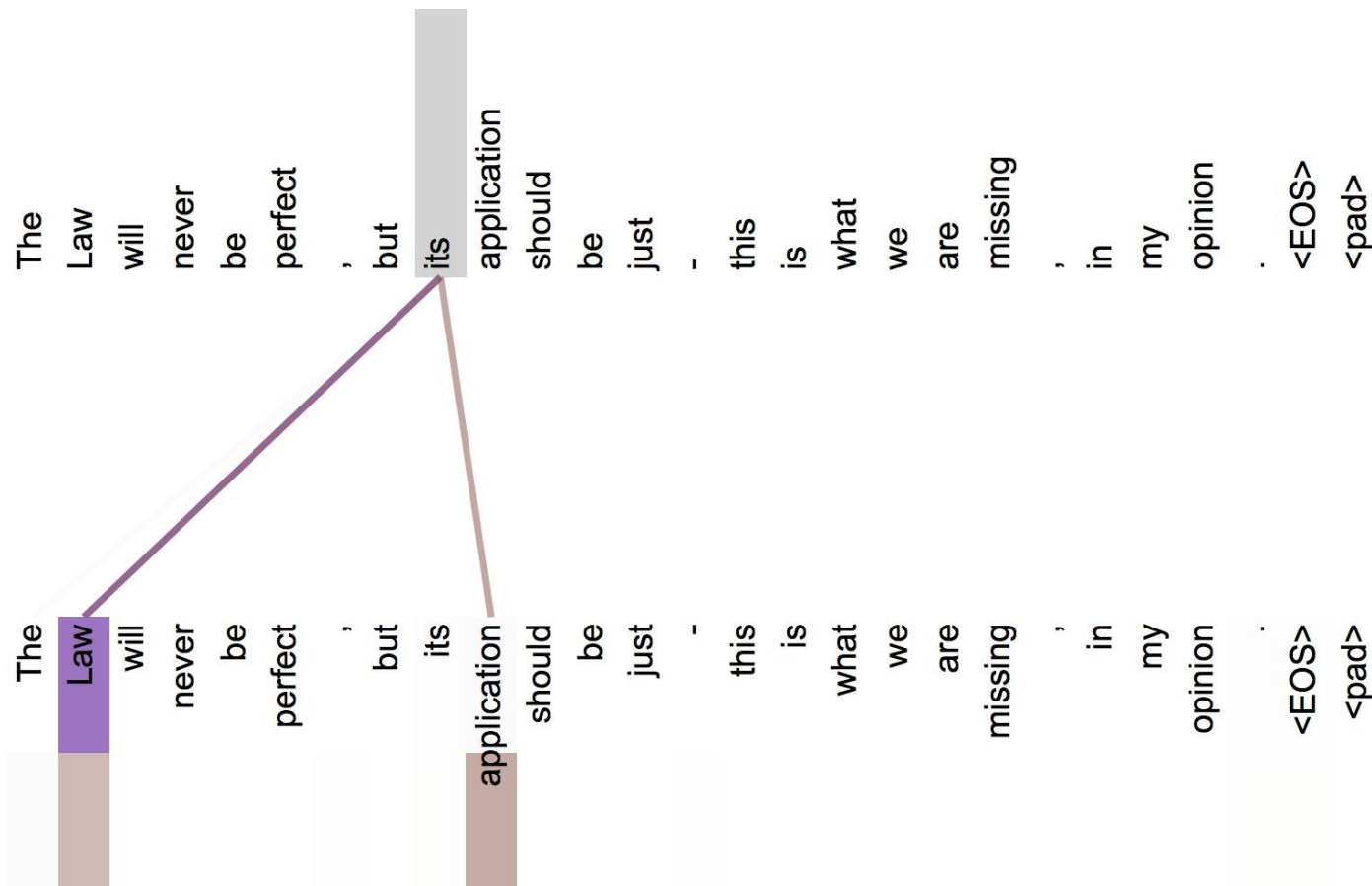
When query term is
“it”

- **One attention head** focuses on “animal”.
 - “it * animal” has the highest score.
- **Another head** focuses on “tired”.

As we encode the word “it”, one attention head is focusing most on “the animal”, while another is focusing on “tired” – in a sense, the model’s representation of the word “it” bakes in some of the representation of both “animal” and “tired”.

Attention visualization: Implicit anaphora resolution

Words start to pay attention to other words in sensible ways.



In 5th layer. Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

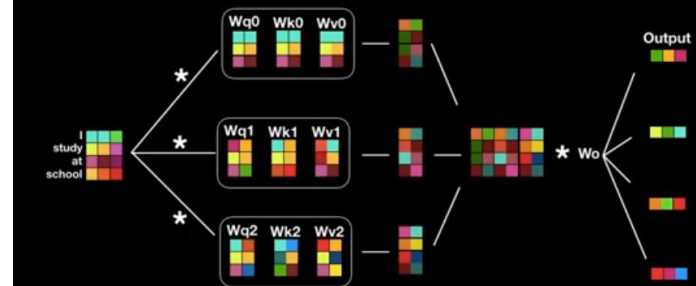
Multi-head attention (equation)

- First map Q, K, V (i.e., input X) into $h=8$ many lower dimensional spaces via W matrices. Then apply attention.
 - $Q, K, V = X$ [seq_len x d_x] ; E.g., [4 x 3]
 - $Q_i = QW_{qi}$ E.g., [4 x 3] [3 x 2] = [4 x 2]
 - $K_i = KW_{ki}$
 - $V_i = V W_{vi}$ E.g., [4 x 2]
 - $head_i = Attention(Q_i, K_i, V_i)$
- Then concatenate outputs and pipe through linear layer.

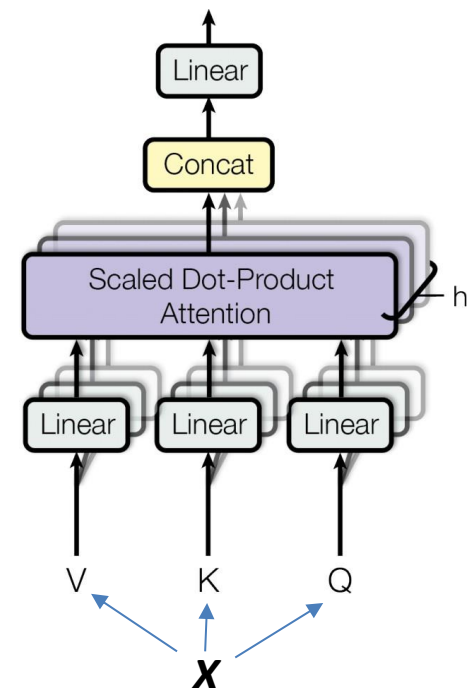
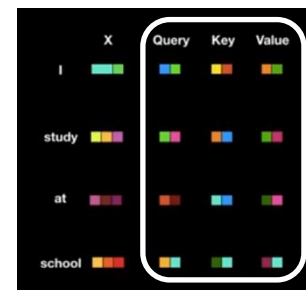
E.g., [4 x 16] [16 x 3] = [4 x 3]

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

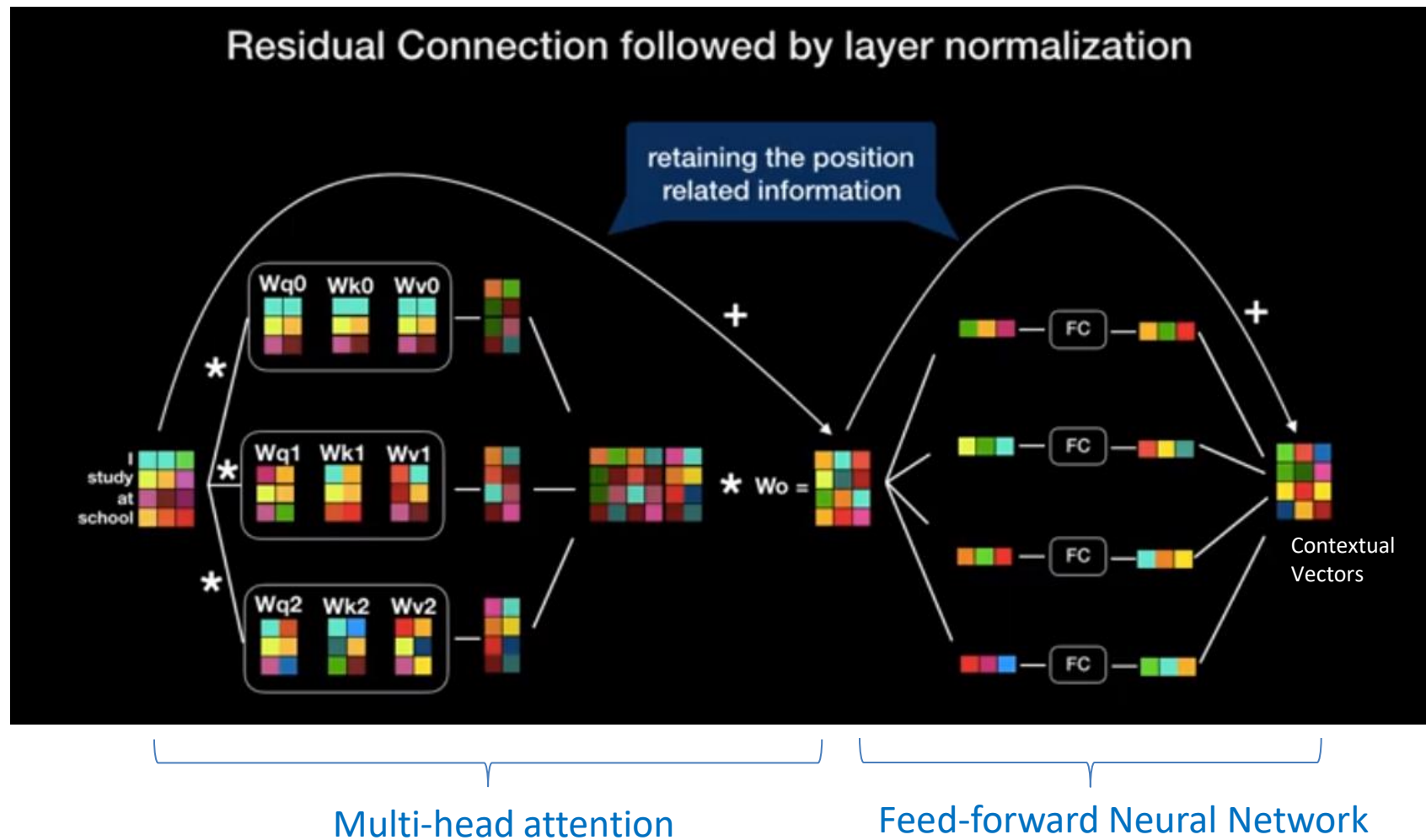
Multi Head Attention



reference: <http://jalamar.github.io/illustrated-transformer/>



Encoder



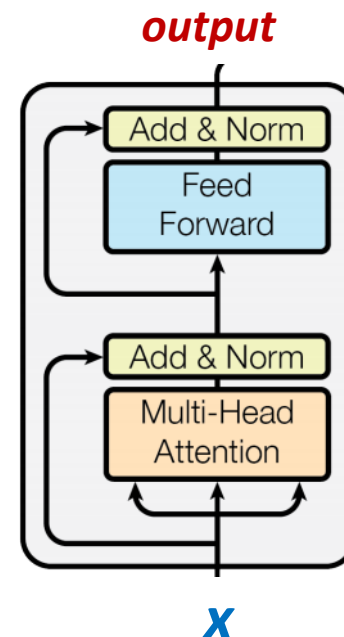
Complete transformer block

Each block has two “sublayers”

1. Multihead attention
2. 2-layer feed-forward NNet (with ReLU)

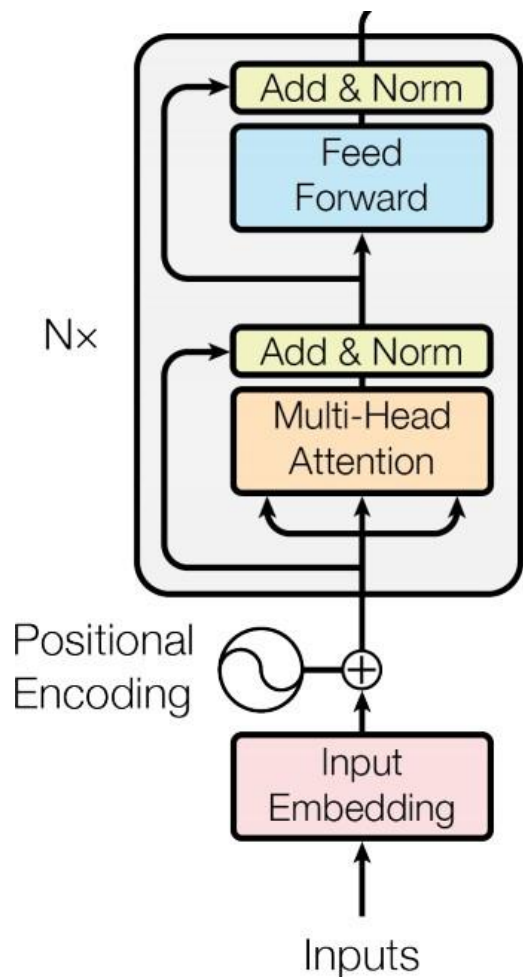
Each of these two steps also has:

- Residual (short-circuit) connection and LayerNorm
 - $\mathbf{x} = \text{LayerNorm}(\mathbf{x} + \text{Multi-Head Attention}(\mathbf{x}))$
 - $\text{output} = \text{LayerNorm}(\mathbf{x} + \text{Feed Forward}(\mathbf{x}))$
- Layernorm changes input features to have mean 0 and variance 1 per layer.



Complete Encoder

- Blocks are repeated 6 or more times
 - (in vertical stack)



Encoder Layer 6

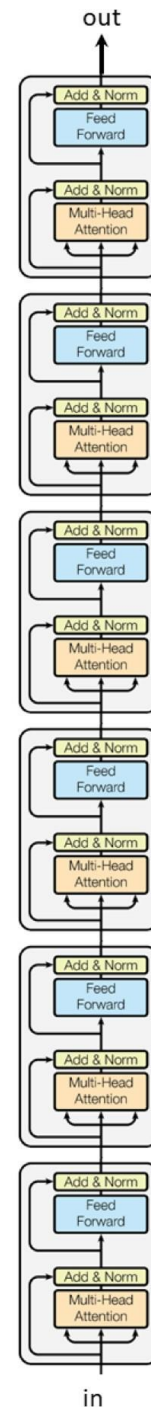
Encoder Layer 5

Encoder Layer 4

Encoder Layer 3

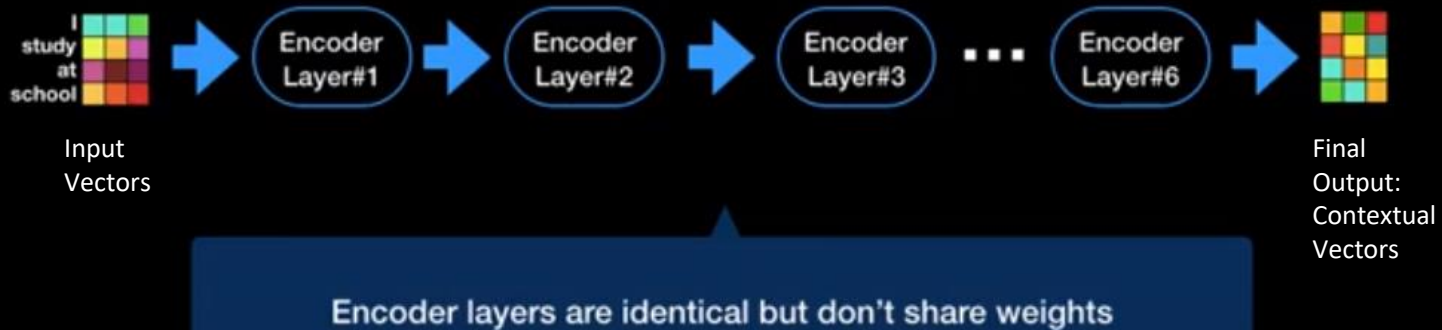
Encoder Layer 2

Encoder Layer 1



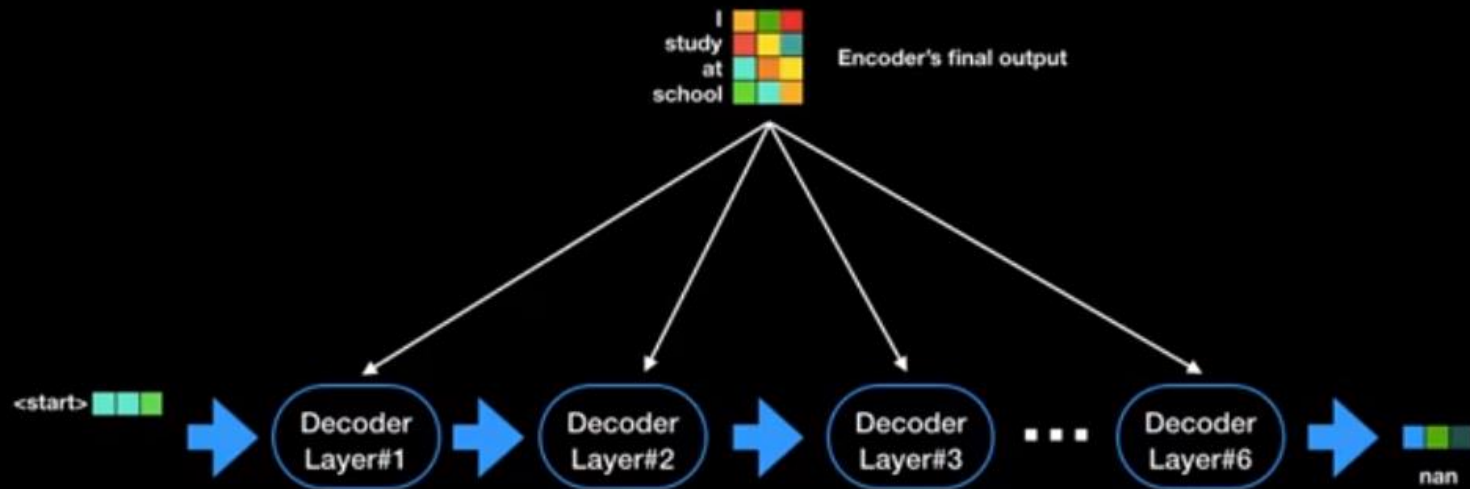
Encoder

Transformer has 6 encoder layers



Decoder

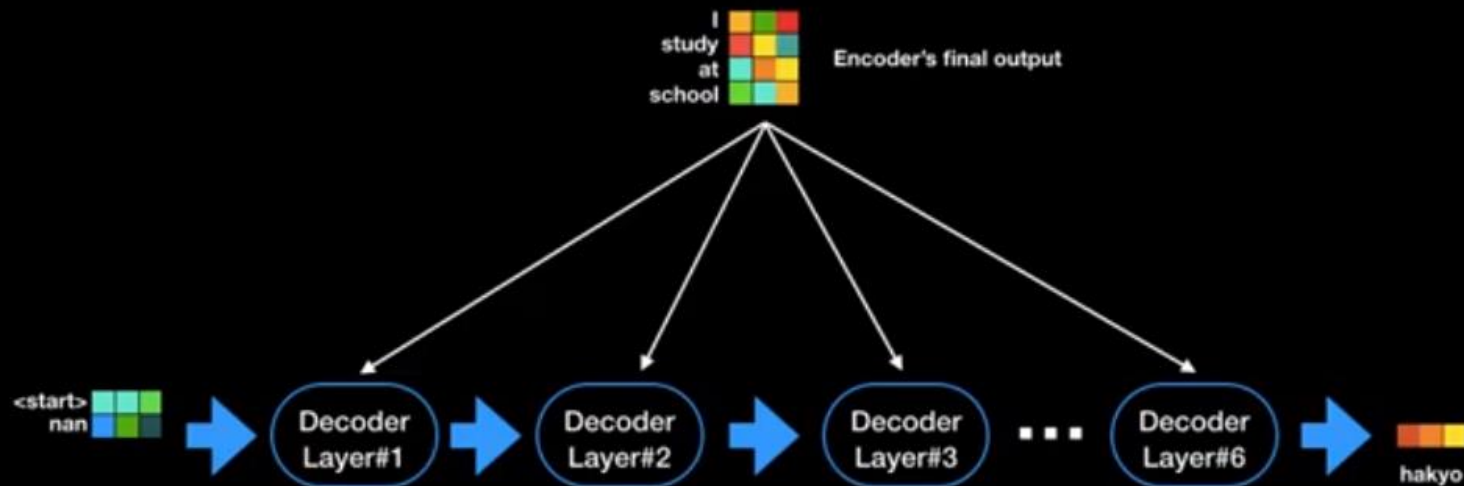
Transformer has 6 decoder layers



The decoder generates one word at a time from left to right.
The first word is based on the final representation of the encoder

Decoder

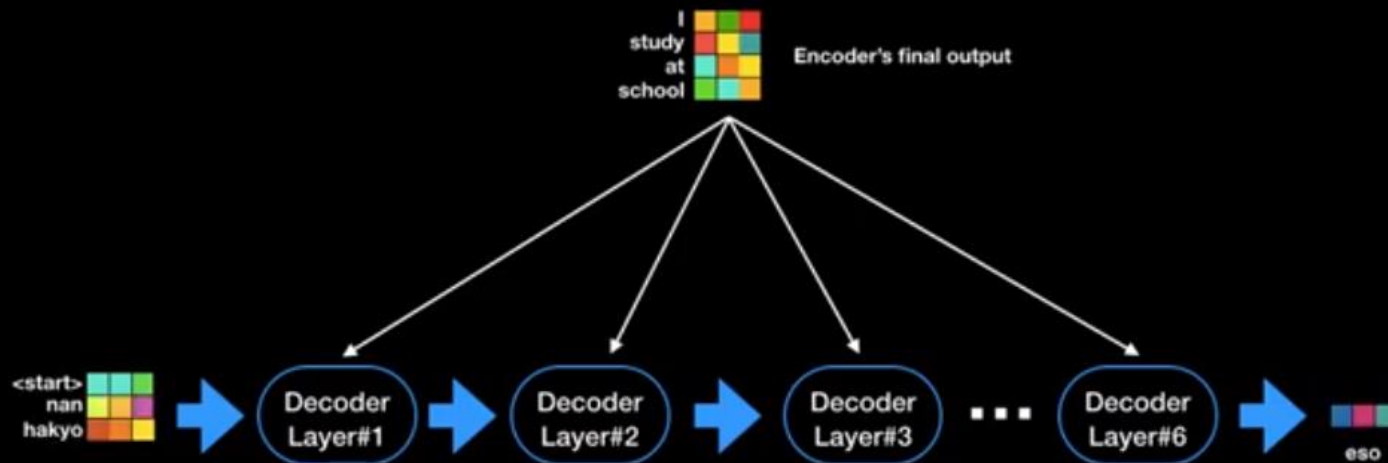
Transformer has 6 decoder layers



Decoder layers attend previously generated words of the decoder and the final output of the encoder

Decoder

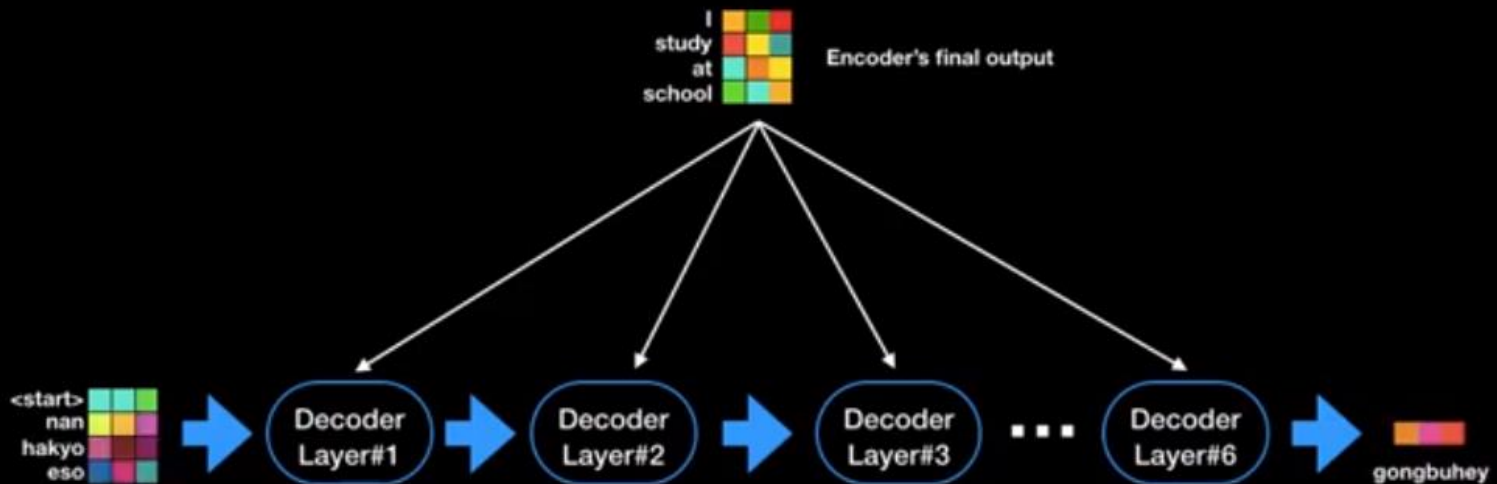
Transformer has 6 decoder layers



Decoder layers attend previously generated words of the decoder and the final output of the encoder

Decoder

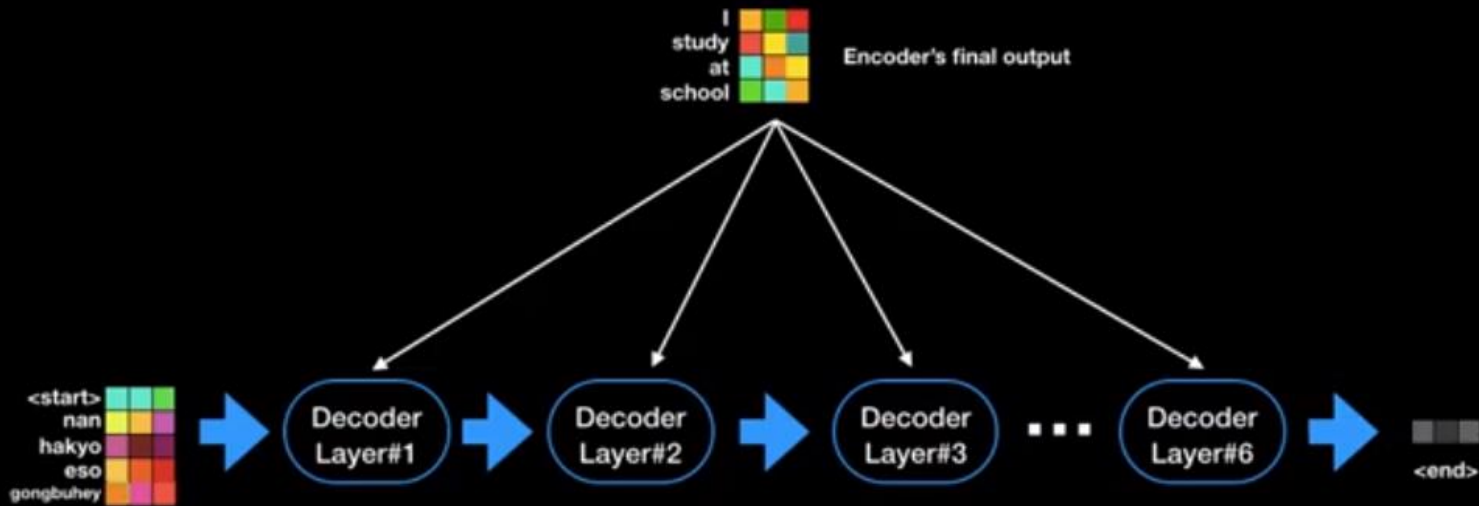
Transformer has 6 decoder layers



Decoder layers attend previously generated words of the decoder and the final output of the encoder

Decoder

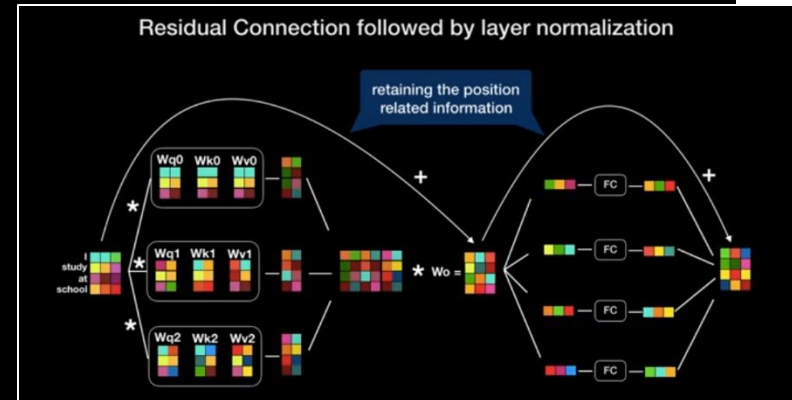
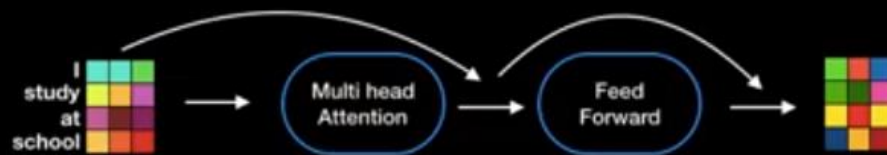
Transformer has 6 decoder layers



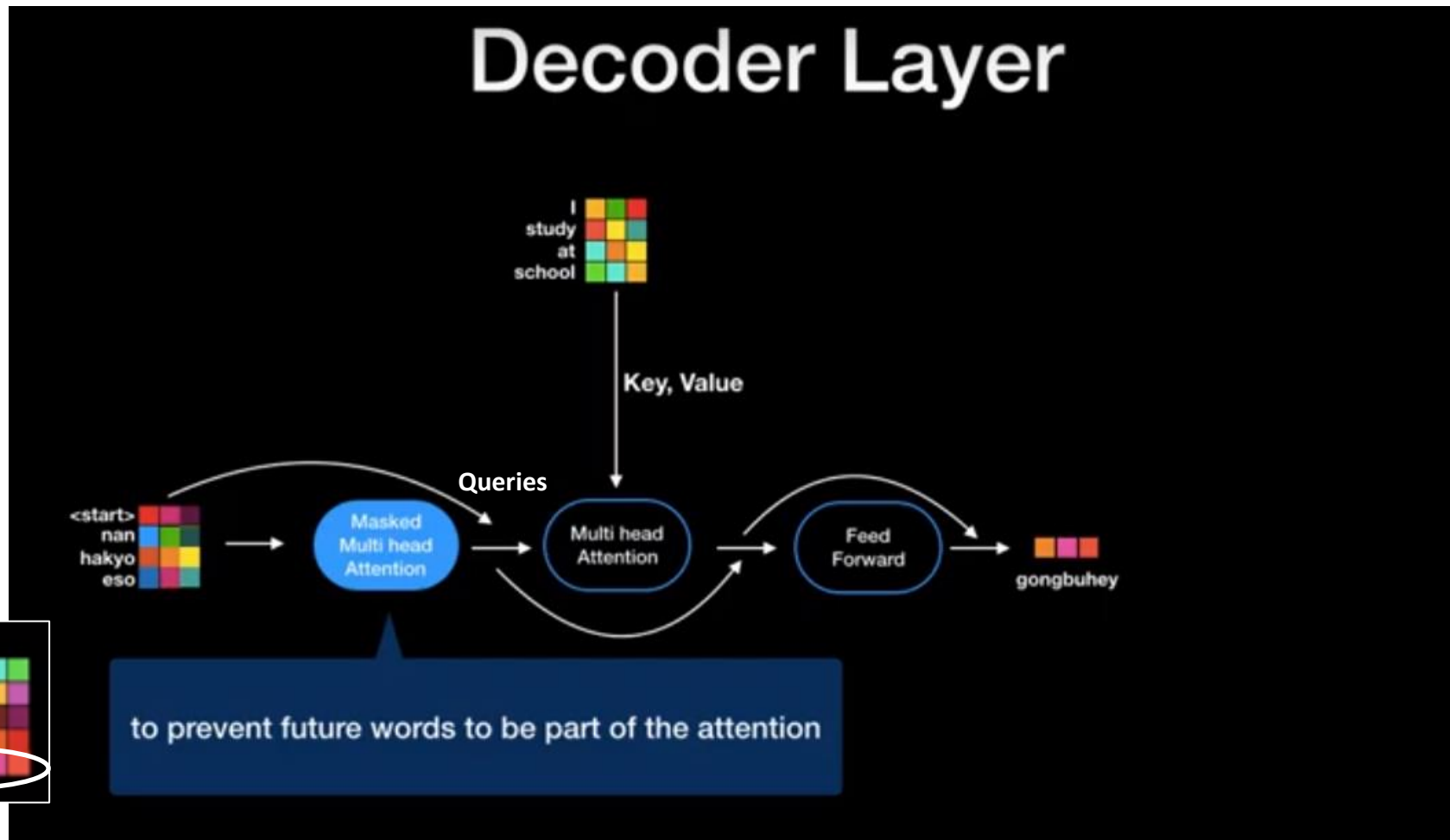
Decoder layers attend previously generated words of the decoder and the final output of the encoder

Encoder

Encoder Layer

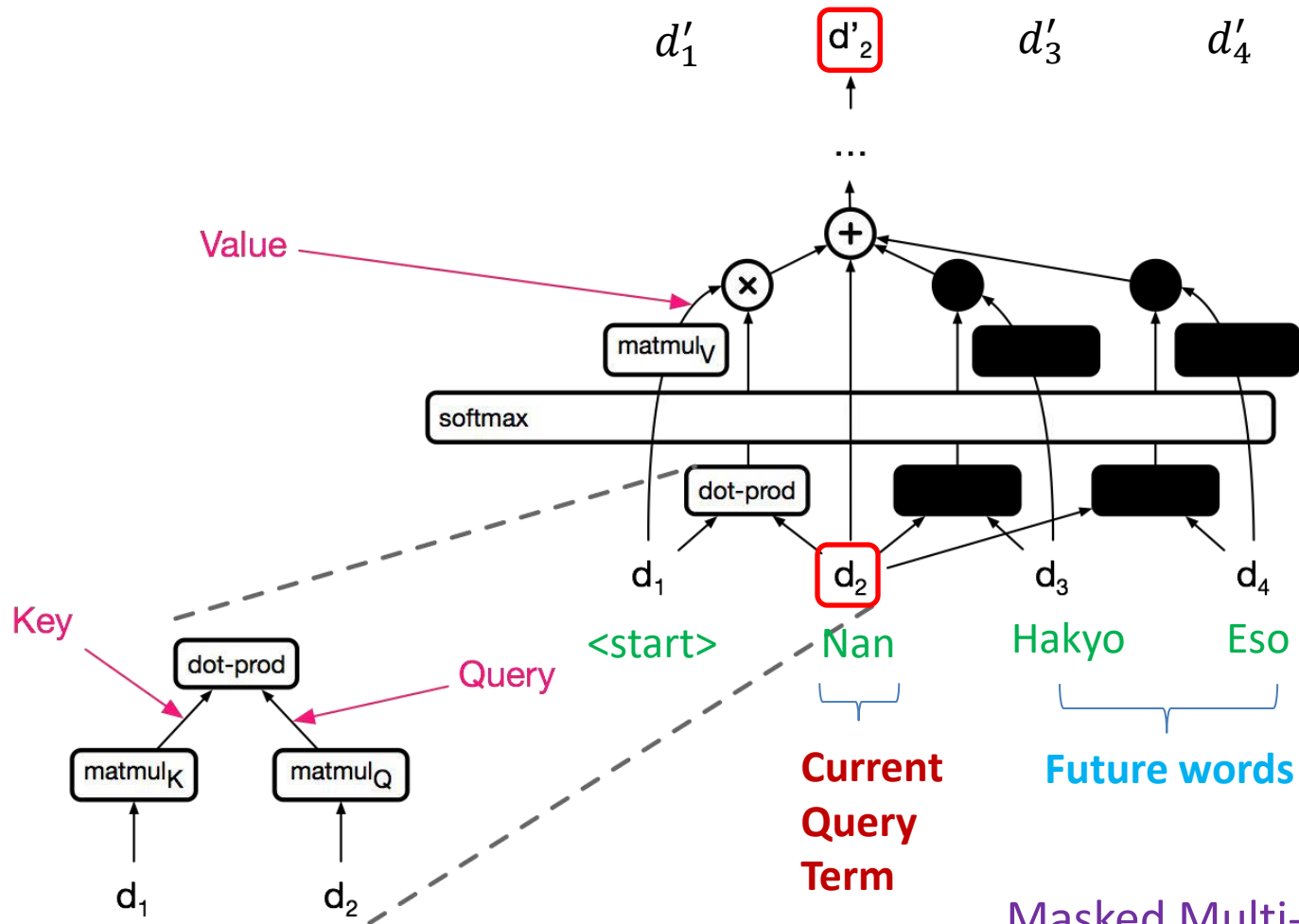


Decoder



Ignore "gongbuhey" in training data.

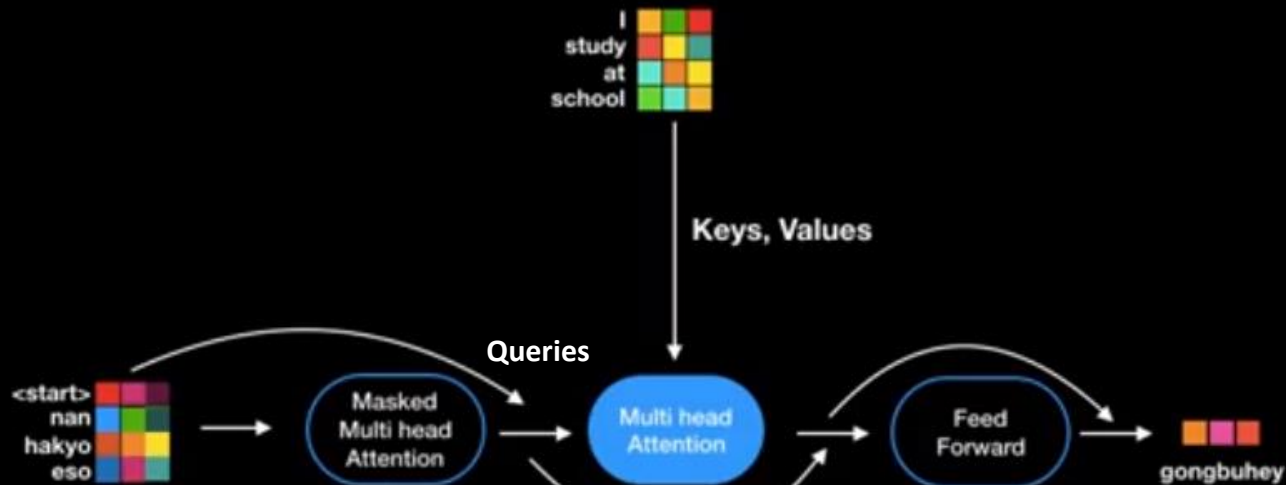
Decoder Self-Attention



Masked Multi-Head Attention:
prevent future words to be part
of the attention.

Decoder

Decoder Layer

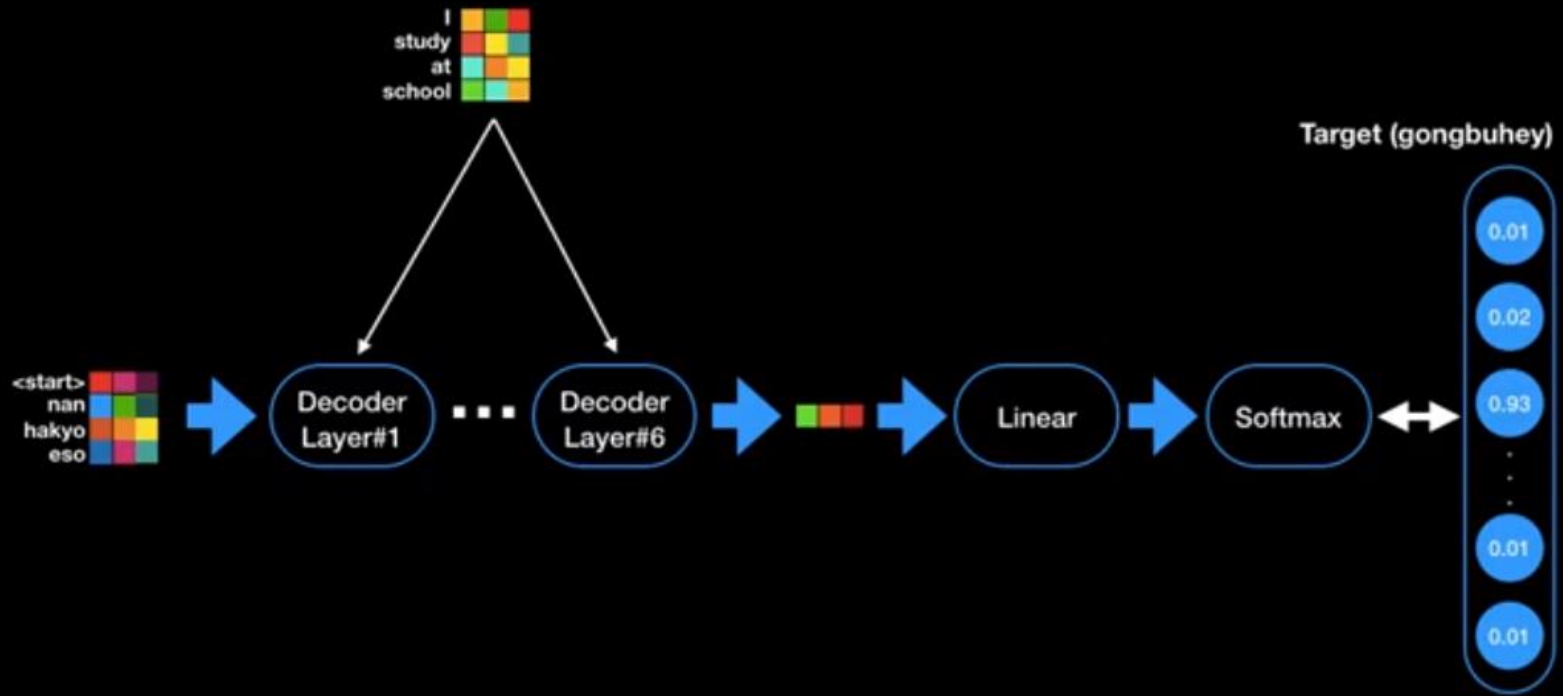


the queries come from the previous decoder layer, and the keys and values come from the output of the encoder.

MultiHead(Q, K, V)

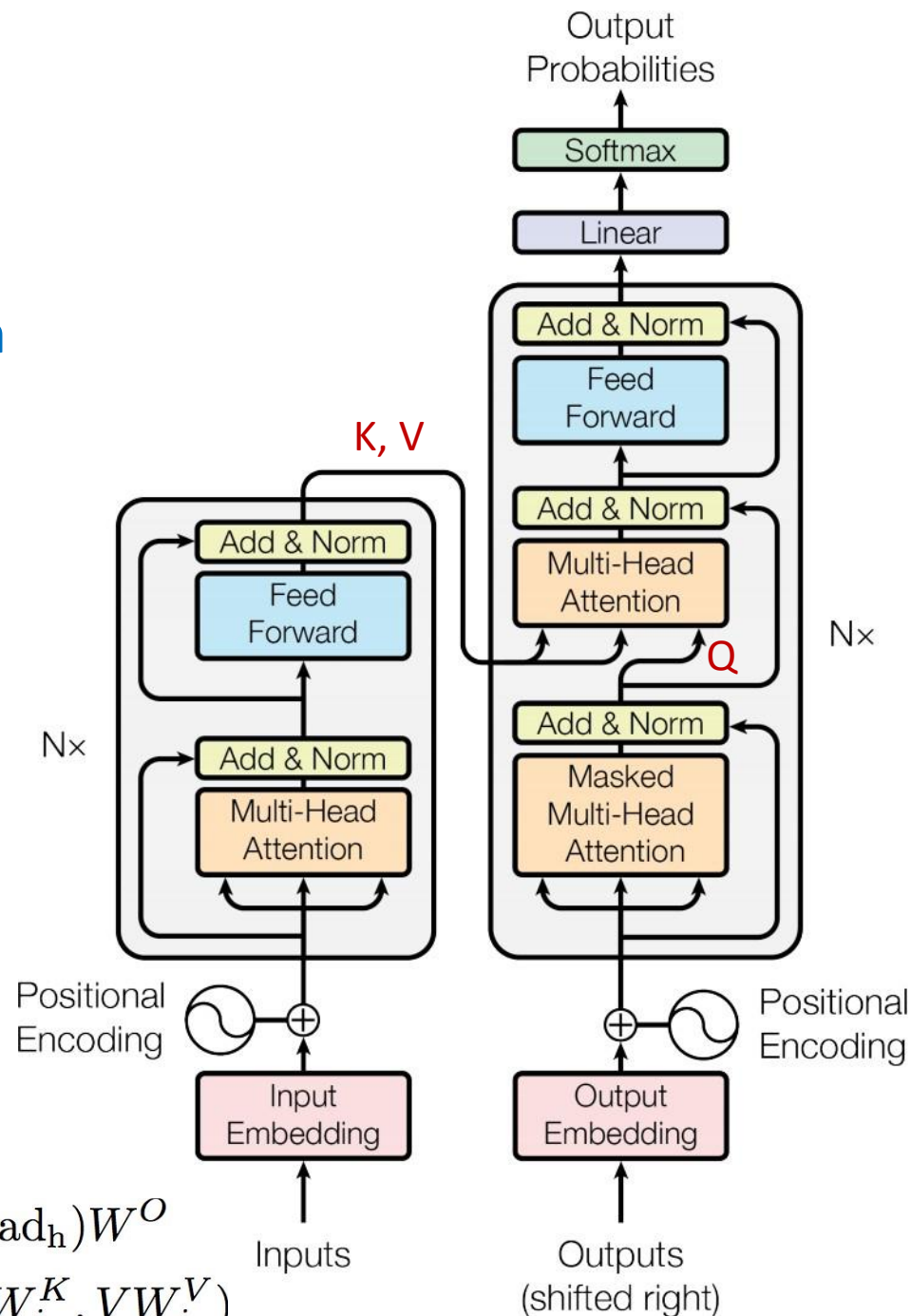
Decoder

How Transformer translate the final output vector?



Transformer Decoder

- 2 sublayer changes in decoder.
- **Masked decoder self-attention** on previously generated outputs.
- **Encoder-Decoder Attention**, where queries come from previous decoder layer and keys and values come from output of encoder.
- Blocks repeated 6 times also.



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Experimental Results for MT

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Transformer-based models

All of these models are Transformer-based architecture models

ELMo
Oct 2017
Training:
800M words
42 GPU days



GPT
June 2018
Training
800M words
240 GPU days



BERT
Oct 2018
Training
3.3B words
256 TPU days
~320–560
GPU days



GPT-2
Feb 2019
Training
40B words
~2048 TPU v3
days according to
[a reddit thread](#)



GPT-3, GPT-4,
ChatGPT, XL-
Net, ERNIE,
Grover, RoBERTa,
T5, etc.
July 2019—



BERT: Devlin, Chang, Lee, Toutanova (2018)

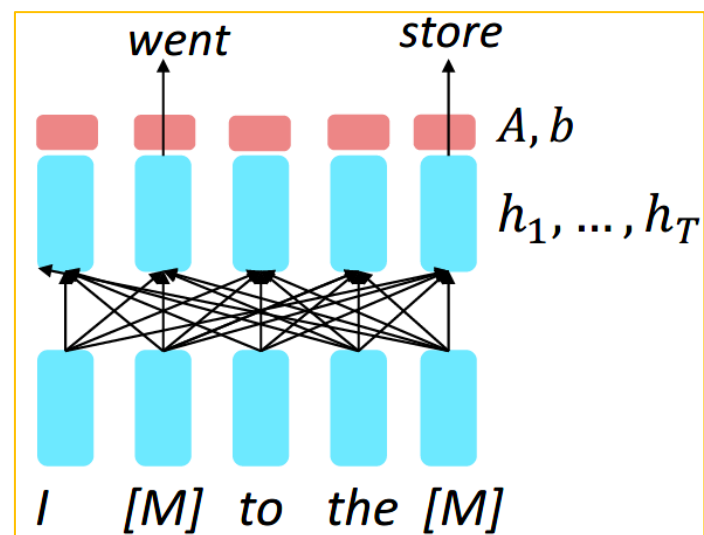
BERT (Bidirectional **Encoder** Representations from Transformers):

Pre-training of Deep Bidirectional Transformers for Language Understanding, which is then **fine-tuned** for a task.

Want: truly bidirectional information flow without leakage in a deep model.

Solution: Use a **cloze task** formulation where 15% of words are blanked out and predicted (**Masked LM**):

Original sentence: “I went to the store.”



Use only the **encoder** of Transformer

Next Sentence Prediction (NSP): Given a pair of two sentences, the task is to say whether the second follows the first (binary classification). E.g., S1: the man went to the store to buy a gallon of milk.

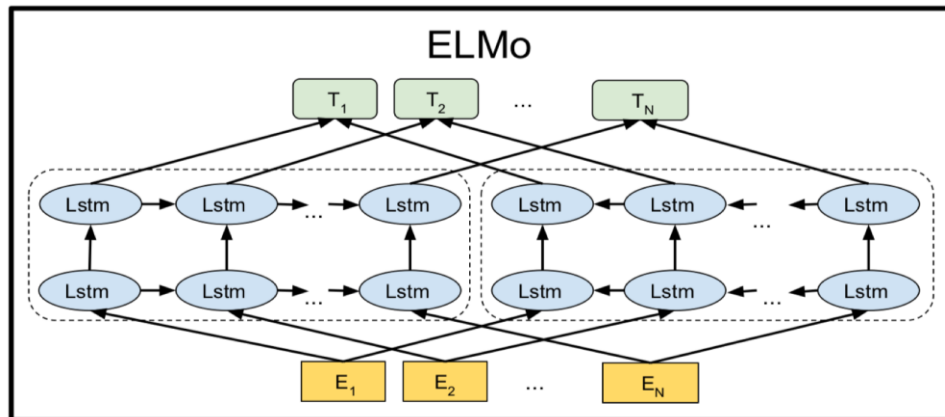
S2: He bought candy too. (True)

Vs. UK airline is testing digital health pass. (False)

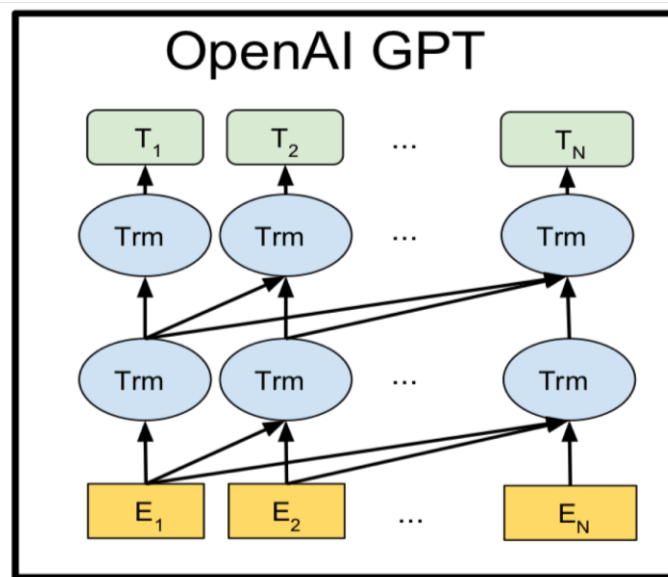
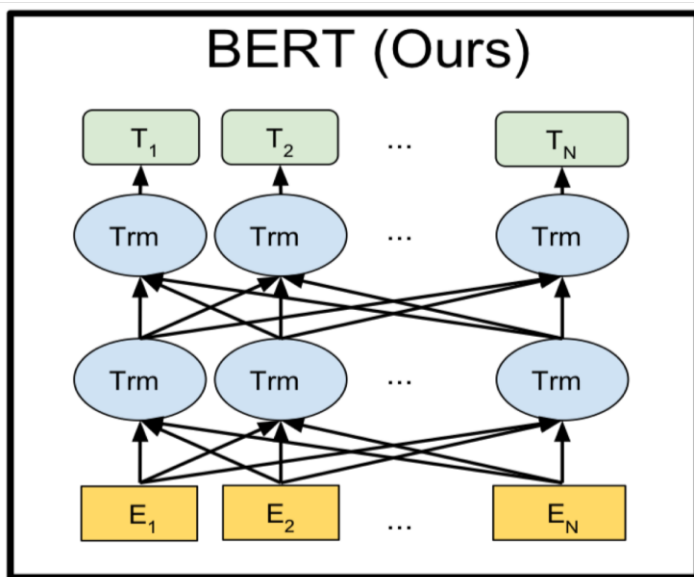
BERT: What kinds of things does pretraining learn using Masked LM?

- *Stanford University is located in ____, California.* [Trivia; Stanford]
- *I put ____ fork down on the table.* [syntax; the]
- *The woman walked across the street, checking for traffic over ____ shoulder.* [coreference; her]
- *I went to the ocean to see the fish, turtles, seals, and ____.* [lexical semantics/topic; dolphins]
- *Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was ____.* [sentiment; bad]
- *Iroh went into the kitchen to make some tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the ____.* [some reasoning – this is harder; kitchen]
- *I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, __* [some basic arithmetic; they don't learn the Fibonacci sequence; 34]

BERT: Devlin, Chang, Lee, Toutanova (2018)



- Bi-LSTM-based model.
- Word embeddings are context-sensitive.



- Use only the **encoder** of Transformer.
- Gets bidirectional context – can condition on future!
- Use only the **decoder** of Transformer.
- Good for language generation, but cannot condition on future words

BERT model architecture and training

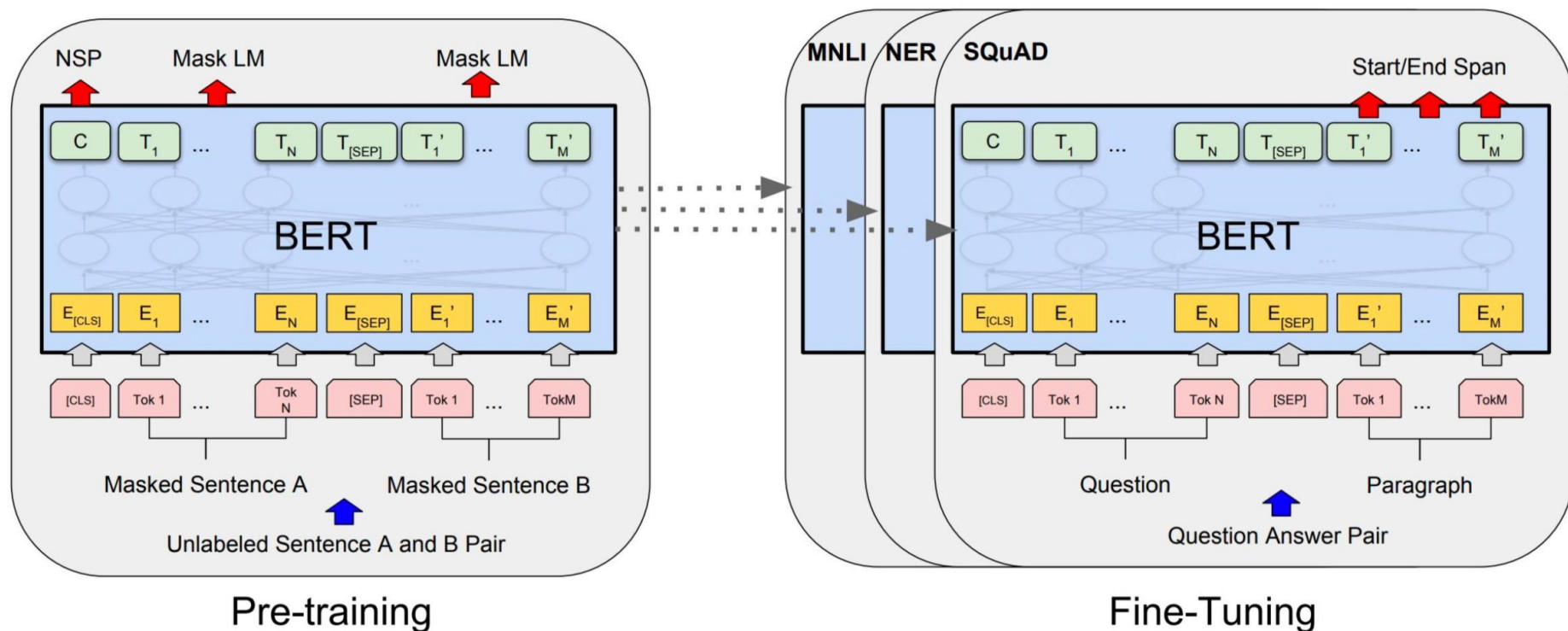
- Use only the **encoder** of Transformer.
- Self-attention $\Rightarrow$ no locality bias
 - Long-distance context has “equal opportunity”.
- Single multiplication per layer $\Rightarrow$ efficiency on GPU/TPU

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Trained on Wikipedia (2,500M words) + Toronto Book Corpus (800M words)
- Train 2 model sizes:
 - BERT-Base: 12-layer (i.e., 12 encoders), 768-hidden, 12-head
 - BERT-Large: 24-layer (i.e., 24 encoders), 1024-hidden, 16-head
- Trained on 4x4 or 8x8 TPU slice for 4 days.

BERT model fine tuning

- Simply learn a classifier built on the top layer for each task that you fine tune for.

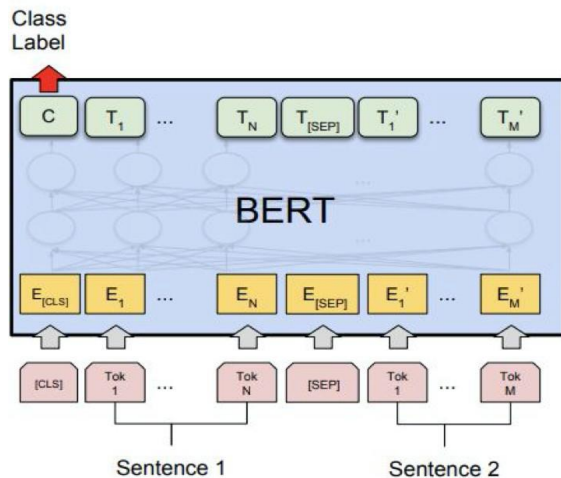


Next Sentence Prediction (NSP)

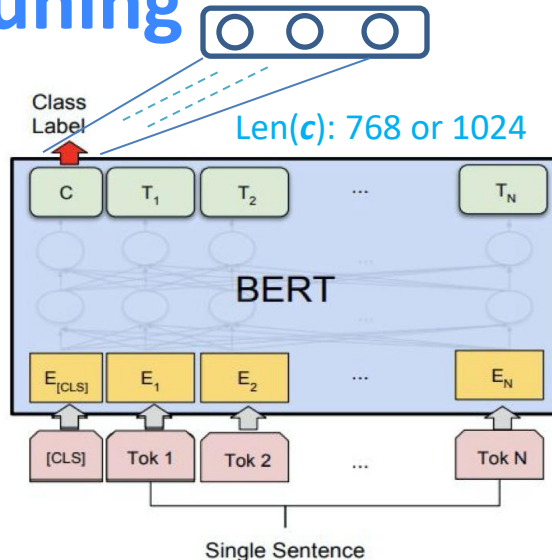
Given a pair of two sentences, the task is to say whether the second follows the first (binary classification).

BERT model fine tuning

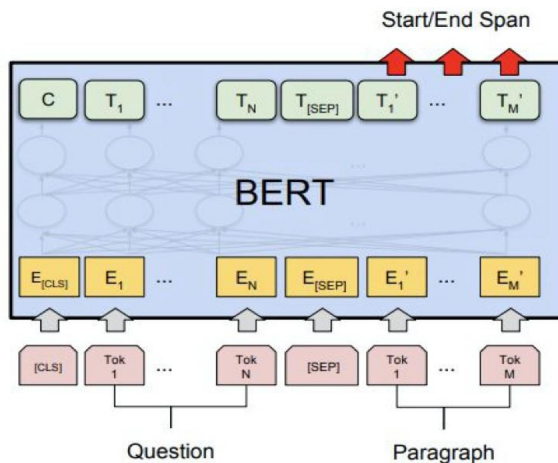
Pos, Neu, Negative class



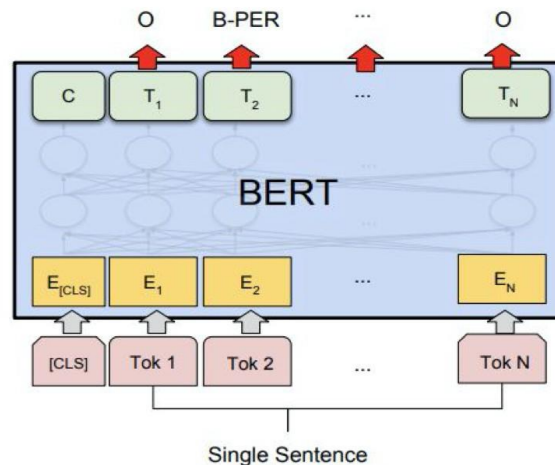
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

MNLI: Given a pair of sentences, predict whether the second sentence is an entailment (can be inferred), contradiction, or neutral with respect to the first one.

QQP: Quora Question Pairs (whether semantically equivalent)

QNLI: natural language inference over question

answering data (The positive examples are (question, sentence) pairs which do contain the correct answer)

STS-B: semantic textual similarity

MRPC: Microsoft paraphrase corpus

RTE: a small natural language inference corpus

SWAG: grounded commonsense inference

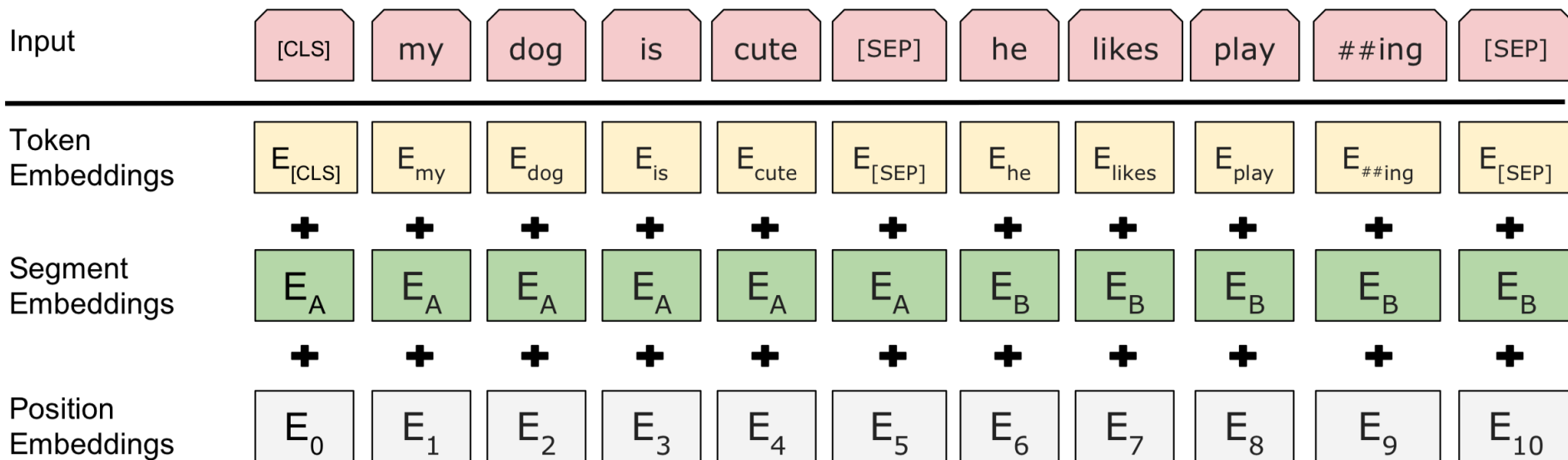
SST-2: sentiment analysis

CoLA: corpus of linguistic acceptability (detect whether sentences are grammatical)

BERT sentence pair encoding

Sentence 1: my dog is cute

Sentence 2: he likes playing



- Token embeddings are word pieces.
- Segment embeddings separate multiple inputs.
- Position embeddings are for word positions.

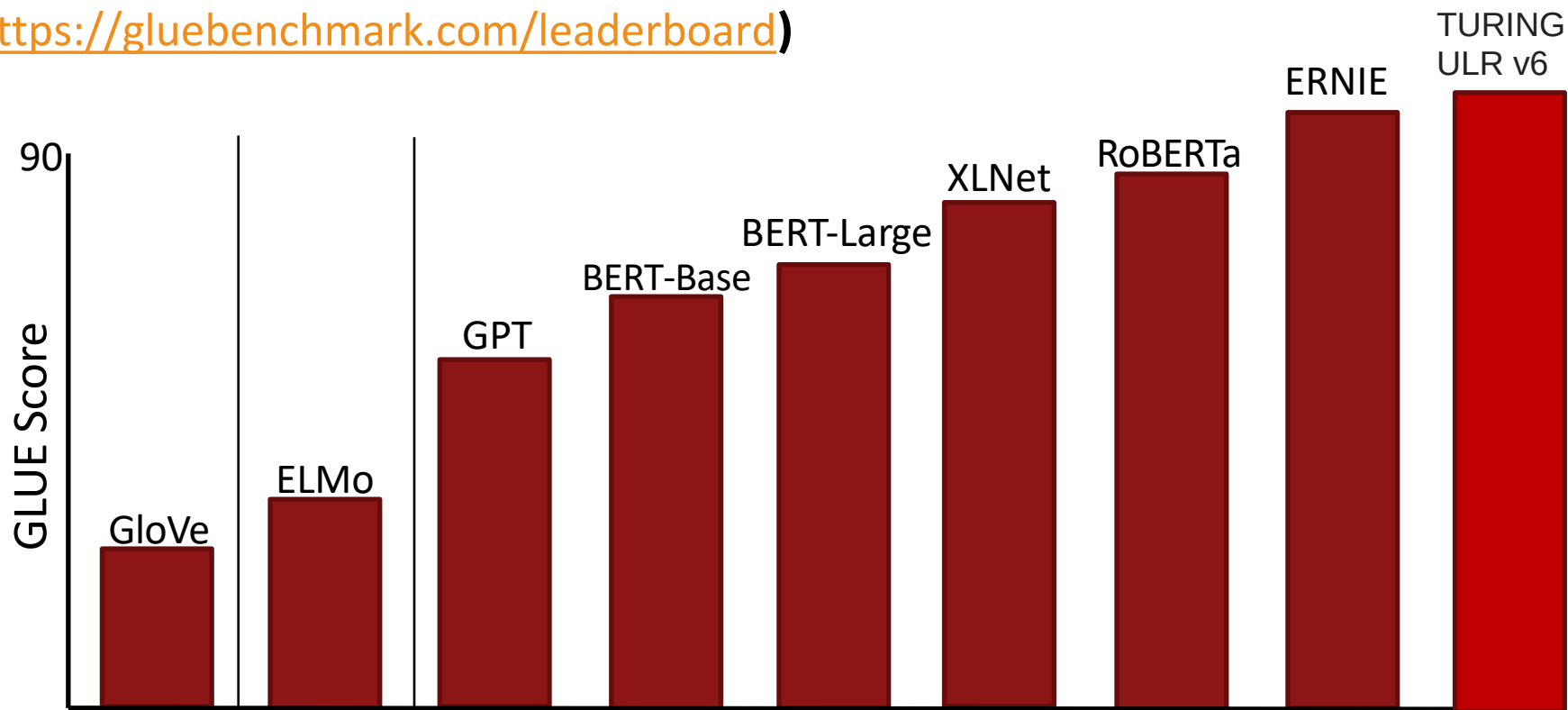
Wordpiece model

- Wordpiece model tokenizes inside words.
- BERT uses a variant of the wordpiece model.
 - (Relatively) common words are in the vocabulary:
 - *hat, learn, at, fairfax, 1910s*
 - Other words are built from wordpieces:
 - *340m* = *340 ##m*
 - *playing* = *play ##ing*
 - *laern* (misspelled word) = *la ##ern*
 - *Hypatia* (person name) = *h ##yp ##ati ##a*
- This approach helps to handle unknown words.
 - Otherwise, “*340m*”, “*laern*”, and “*hypatia*” are represented as <unknown> word.

How's the weather?

Rapid Progress from Pre-Training (GLUE benchmark:

<https://gluebenchmark.com/leaderboard>)



Over 3x reduction in error in recent years, “superhuman” performance

The General Language Understanding Evaluation (GLUE) is a benchmark of nine sentence- or sentence-pair language understanding tasks built on established existing datasets. (<https://gluebenchmark.com/>)

Referenced Materials

- Natural Language Processing with Deep Learning,
<http://web.stanford.edu/class/cs224n/>
- Transformer, Minsuk Heo,
<https://www.youtube.com/watch?v=z1xs9jdZnuY>